ShuffleSnap: strided windowed exact reassignment for point-to-grid layout, evaluated against certified optima

Draft — author list to be decided

September 2026

Abstract

Snapping each point of a 2D embedding (such as a UMAP layout) to its own cell of a grid, while minimizing total squared displacement, is a linear assignment problem. Dense solvers cannot handle it beyond a few tens of thousands of points. We develop and test ShuffleSnap, a local search inspired by Shellsort. At a decreasing sequence of strides, it splits the grid into interleaved subgrids and solves small exact assignment problems in shifted 6×6 windows of each.

To measure true optimality gaps, we built a certified exact solver. It combines min-cost flow on sparse candidate edges with a Lagrangian lower bound priced over all cells. Started from ShuffleSnap, it proved optimality on every instance we tried, up to 10^6 points.

On held-out instances with 10^5 points, the baseline as specified is within 0.2% of the optimum on nine of eleven distributions. It takes about 5 s on one core and 0.35 s on 24 threads. A finer stride sequence (ratio $\sqrt{2}$) with 8×8 windows reduces the excess a further 2.6–14× on ten of the eleven. BSP-OT is faster but typically 10–350× further from optimal. RasterFairy is 17–425% above optimal on most inputs. A dense exact solver already needs up to 6 minutes at 10^4 points.

The method's limits are structural. Its error consists of long closed loops of one-cell shifts that no window can remove, and we construct inputs on which its relative gap is unbounded. Uniform points (2.5% at 10^5, 5.3% at 10^6), jittered lattices (33%), a concave mask (18%, worse than BSP-OT) and surplus cells remain hard. Structured starts make results worse, and polishing at coarser strides barely changes the result. Warm-started from ShuffleSnap, the certified solver reaches proven optima in seconds to minutes at 10^5 points, which narrows the practical case for the heuristic.

On real UMAP embeddings, displacement and neighbourhood preservation diverge. BSP-OT is within 2–7% of the optimal displacement but halves the fraction of preserved grid neighbours. We release both the implementation and all experiment code and raw results.

1 Introduction

A common way to show a large image or document collection is to embed the items in the plane (for example with t-SNE [44] or UMAP [29]) and then snap every item to its own cell of a regular grid, so that thumbnails tile the screen without overlap. Snapping is an assignment problem: each point must receive a distinct cell, and we would like points to move as little as possible. Measured as total squared displacement, the best snapping is a linear assignment problem (LAP), and also a discrete optimal-transport problem with quadratic cost [34].

For a few thousand points, practitioners solve this LAP exactly with a dense Jonker–Volgenant or Hungarian solver [19, 26–28]. The dense cost matrix needs $8N^2$ bytes, which is 80 GB at $N =$

100,000, so these solvers stop being usable somewhere between 10,000 and 30,000 points. Beyond that, people turn to recursive-splitting heuristics such as RasterFairy [21] or to fast approximate transport such as BSP-OT [12].

This paper develops and evaluates ShuffleSnap, a simple heuristic for this regime, taking its name from Shellsort [41]. Shellsort sorts interleaved subsequences at a decreasing sequence of gaps. ShuffleSnap does the same in 2D: at stride s it splits the grid into s^2 interleaved subgrids and solves small *exact* assignment problems inside 6×6 windows of each subgrid, halving s until it reaches 1 (Figure 1). All work is at full resolution; nothing is aggregated or downsampled.

We treat the idea as a hypothesis to test, not a result to promote. Our aim throughout was to find where it fails. The main contributions are:

- **A careful implementation** of the specified baseline, with documented boundary, rectangular-grid, surplus-cell and mask rules. It is a parallel C++ kernel with a Python API whose results do not depend on the number of threads.
- **A certified exact solver that reaches 10^6 points.** It solves min-cost flow on sparse candidate edges, then prices every edge of the *dense* problem through a Lagrangian bound, adding edges until the bound meets the primal cost. It gives true optimality gaps, not progress relative to our own best answer, for all 211 benchmark instances (up to 200,000 points) and for uniform instances up to 10^6 points. It turned out to be a strong practical baseline as well.
- **Measured improvements and non-improvements.** Changes were chosen on a development set and evaluated on held-out instances. A finer stride sequence (ratio $\sqrt{2}$ instead of 2) with 8×8 windows reduces the excess over the optimum 2.6–14×, at about three times the running time. Several intuitive ideas did not help:
  - sorted or bisection starts make results *worse*;
  - polishing at coarser strides and merging independent runs help little or not at all;
  - a Jonker–Volgenant window solver is slower.
- **A diagnosis of the remaining error.** The gap to the optimum is concentrated in a few long closed loops of points, each displaced by about one cell. No window smaller than a loop can remove it. Masks with concave bays and grids with surplus cells expose the same mechanism at a larger scale.
- **A benchmark against exact LAP, RasterFairy and BSP-OT.** We first verified each comparator's capabilities: we found and worked around coordinate truncation in the PyPI release of RasterFairy, and used its much stronger upstream version. The benchmark covers varied and adversarial distributions, masks, prime point counts, two real UMAP embeddings with 70,000 points each, multiple seeds, matched-budget curves, and scaling to 10^6 points. We report cost, time, peak memory, and neighbourhood preservation, and we keep displacement and neighbourhood measures separate.

The implementation (`shufflesnap`) and this paper's experiment code, raw results and figure scripts are released as two separate repositories.

2 Problem

Cells. A target is a boolean mask on a $W \times H$ lattice. Lattice position (x, y) is column x and row y, and usable positions are the cells $c_1, \ldots, c_M$. Cell centers default to the integer coordinates, so

one unit is one cell pitch. Anisotropic pitches (e.g. for 4:3 thumbnails) scale the centers but leave the lattice unchanged. A full rectangle, a rectangle whose last row is only partly usable, a disk, a ring, and the shape of a letter are all masks.

Points and normalization. Input points $q_1, \dots, q_N$ ($N \le M$) are mapped into the cell frame before anything else. Unless stated otherwise, each axis of the point bounding box is mapped affinely onto the bounding box of the usable cell centers (*bbox* normalization); p_i denotes the mapped points. The mapping is part of the problem: in every comparison, all methods receive identical p_i and identical cell centers.

Objective. We seek an injective assignment $\sigma : \{1..N\} \to \{1..M\}$ minimizing

$$C(\sigma) = \sum_{i=1}^{N} \|p_i - c_{\sigma(i)}\|^2. \tag{1}$$

When $N = M$ this is a bijection. When $N < M$ the solver must also decide which $M - N$ cells stay empty. Unlike the bijective case, the cell term $\sum_i \|c_{\sigma(i)}\|^2$ then depends on the choice, so the surplus case is not just a relabelling of the bijective one.

A certificate of optimality. Equation (1) is the linear program $\min \sum_{ij} c_{ij} x_{ij}$ with $\sum_j x_{ij} = 1$ for every point, $\sum_i x_{ij} \le 1$ for every cell, and $x \ge 0$, where $c_{ij} = \|p_i - c_j\|^2$. Its constraint matrix is totally unimodular. We use the following elementary bound.

Proposition 1. *For any cell potentials $v \in \mathbb{R}^M$,*

$$\mathrm{LB}(v) = \sum_{i=1}^{N} \min_j \big(c_{ij} - v_j\big) + \sum_{j=1}^{M} v_j - (M - N) \max_j v_j$$

is a lower bound on the optimal cost. The bound is tight (equals the optimum) for optimal dual potentials.

Proof. Let $\bar{v} = \max_j v_j$ and $v'_j = v_j - \bar{v} \le 0$. Let $u_i = \min_j (c_{ij} - v'_j)$. Then (u, v') is feasible for the dual $\max \sum_i u_i + \sum_j v'_j$ subject to $u_i + v'_j \le c_{ij}$ and $v' \le 0$, and its objective equals $\mathrm{LB}(v)$ (shifting all potentials by $\bar{v}$ changes every u_i by $+\bar{v}$ and the cell sum by $-M\bar{v}$). Weak duality gives the bound. Strong duality of the LP gives tightness. □

The bound needs a minimum over *all* M cells for every point. It does not depend on how v was obtained, so $C(\sigma) - \mathrm{LB}(v)$ certifies the gap of any assignment σ. We use it for every "excess over optimum" number in this paper, and we never measure progress relative to our own final cost.

Reporting error. We report the *relative excess* $C(\sigma)/C^\star - 1$ and the *absolute excess per point* $(C(\sigma) - C^\star)/N$ in cell units squared, where $C^\star$ is a certified optimum. Both are needed. Well-matched inputs (e.g. a jittered lattice) have a tiny $C^\star$, which makes relative numbers explode. Inputs dominated by outliers (heavy tails) have a huge $C^\star$, which makes every method look perfect in relative terms.

3 Related work

Exact assignment. The Hungarian method [23, 32] and shortest-augmenting-path variants such as Jonker–Volgenant (JV) [18] solve dense $N \times N$ problems in $O(N^3)$ time and $O(N^2)$ memory. SciPy's `linear_sum_assignment` implements a modified JV without initialization [8, 47]. The `lap` and `lapjv` packages provide dense JV with heuristic initialization (`lap` also provides a sparse variant) [19, 27]. Auction algorithms [3, 4] and cost-scaling push–relabel [15] handle sparse problems. The min-cost-flow solver in OR-Tools, which we use, is a cost-scaling push–relabel method [16, 33]. Burkard et al. [7] survey the field. Geometric bipartite matching admits near-linear ε-approximations in theory [38, 40]. We know of no practical implementations aimed at this use case.

Optimal transport. With quadratic cost, optimal maps are gradients of convex functions [6]. Least-squares assignments with prescribed cluster sizes are exactly those induced by power diagrams [1], the geometric fact behind semi-discrete solvers [25, 31] and behind the potentials in our certificate. Multiscale methods solve coarsened problems and refine them [14, 31, 39]. Schmitzer's sparse multiscale algorithm in particular alternates sparse subproblems with global optimality checks [39], as our exact solver does. Entropic regularization [9] and sliced transport [5, 37] trade exactness for speed; see Peyré and Cuturi [34] for a survey. BSP-OT [12] builds many randomized binary space partition matchings in $O(N \log N)$ and merges them with a cycle-based rule. Its official code [13] has Python bindings [10, 43]. The BSP-OT paper does not evaluate uniform grid targets.

Grid layouts for visualization. RasterFairy [21] recursively slices the point cloud and the target raster in matching proportions, and supports masks and hexagonal rasters. Its current upstream version also solves small leaf blocks exactly (Section 6). IsoMatch [11] projects with Isomap and then solves a dense Hungarian problem, which it names as its bottleneck. Kernelized sorting [35, 36], self-organizing maps [22], the Self-Sorting Map [42], and Linear Assignment Sorting and its fast variant FLAS [2] arrange high-dimensional items directly on a grid. LAS repeatedly solves full JV problems between features and smoothed map vectors. Gridfit [20] and DGrid [17] remove overlap in scatterplots by hierarchical partitioning. These methods mostly optimize neighbourhood preservation, and they measure it with trustworthiness and continuity [45, 46] or the Distance Preservation Quality [2]. We keep that goal distinct from displacement: the LAP objective only minimizes how far points move.

Sorting networks and Shellsort. Shellsort [41] sorts with insertion-sort passes over a decreasing sequence of gaps, so early passes move elements far and late passes move them little. ShuffleSnap borrows this schedule. Unlike Shellsort's final gap-1 pass, however, its final stride-1 stage is not guaranteed to reach the optimum. That difference turns out to be the heart of the matter (Section 7.8).

4 ShuffleSnap

4.1 The baseline algorithm

The baseline follows the specification we were given:

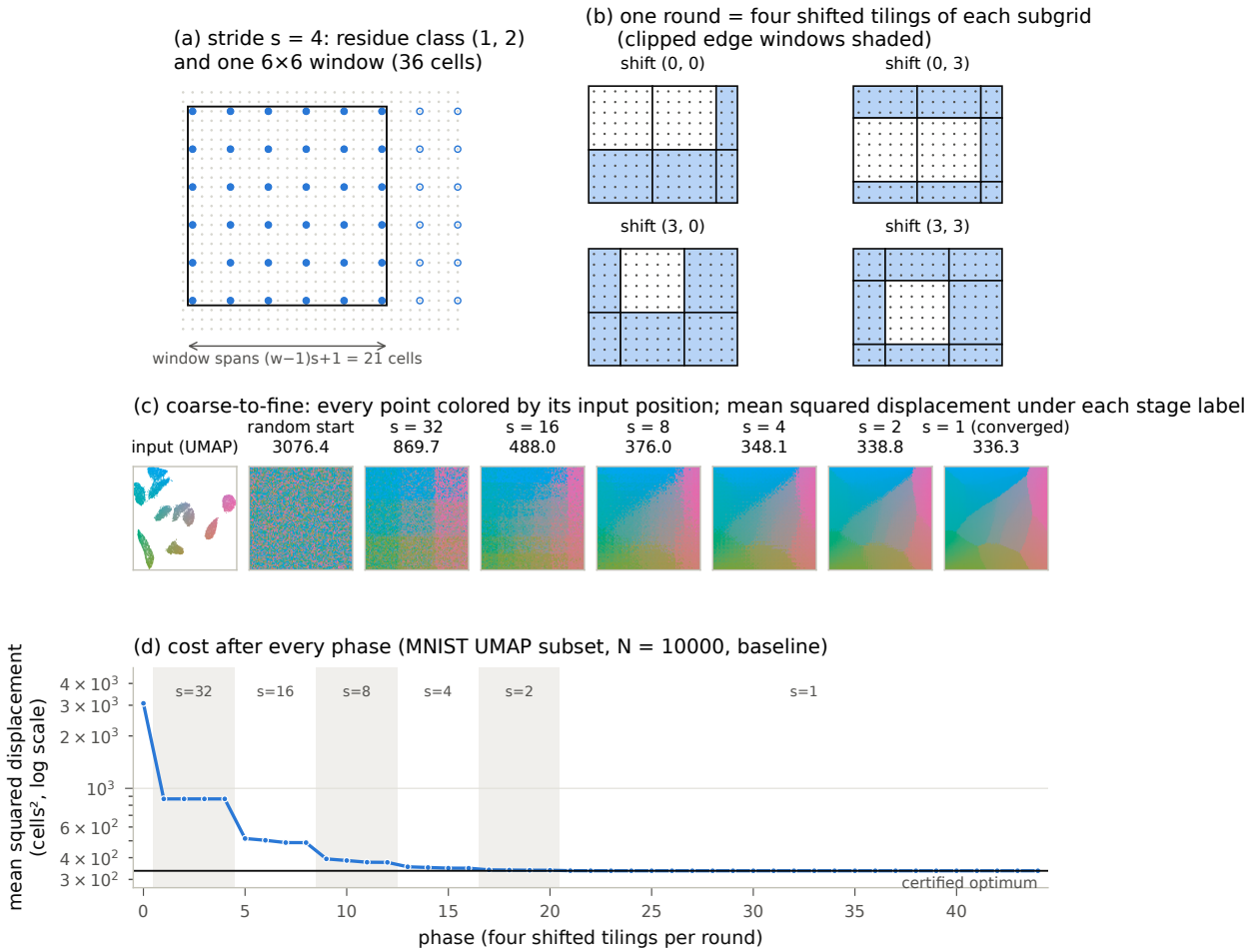


Figure 1: How ShuffleSnap works. (a) At stride $s = 4$ the lattice splits into $s^2 = 16$ interleaved residue subgrids. Blue marks one of them. A 6×6 window of that subgrid (filled blue) holds 36 cells spaced 4 apart and spans 21 cells. The occupants of those 36 cells are reassigned among the same cells by an exact linear assignment. (b) Each round tiles every subgrid four times, with the tiling shifted by half a window in x, y, or both, so that window borders move. Windows that cross the subgrid edge are clipped (shaded), never dropped, on all four sides. (c) Progress on a 10,000-point subset of a UMAP embedding of MNIST. Each point is colored by its *input* position, so a smooth color field means the input neighbourhood structure survived, and seams are visible as color jumps. (d) Mean squared displacement after every phase. The coarse strides remove most of the cost; the stride-1 rounds that follow change little.

1. **Start.** Place the N points on a uniformly random subset of N cells, as a seeded random permutation.
2. **Strided windows.** At stride (s_x, s_y), lattice positions with the same residues (x mod s_x, y mod s_y) form a subgrid, for $s_x s_y$ subgrids in total. In its own coordinates each subgrid is tiled with 6×6 windows. A window therefore contains at most 36 cells, spaced s apart on the lattice and spanning $5s + 1$ cells per axis.
3. **Exact reassignment.** In every window, the points currently occupying its cells are reassigned among the same cells, empty ones included, by solving the dense linear assignment on squared distances to the original (normalized) coordinates. The window's cell set never changes, so the global assignment stays injective.

4. **Shifted tilings.** The tiling is repeated with shifts $(0,0)$, $(0,3)$, $(3,0)$ and $(3,3)$ in subgrid coordinates. These four phases form one round. Windows in one phase are disjoint and are solved in parallel.
5. **Schedule.** On an axis of length L, the first stride is the smallest power of two with $6s \geq L$. After one round, both strides are halved (never below 1). Stride-1 rounds then repeat until a full round moves no point, or until a time budget runs out.

Window solver and acceptance. Each window is a rectangular problem: k occupants and $m \geq k$ cells, with $m \leq 36$ in the baseline. We solve it with the shortest-augmenting-path Hungarian method with potentials, in $O(k^2 m)$ time. A cheap warm start does not change the optimum: row reduction, then matching every row whose cheapest column is still free. It saves 5–20% in a micro-benchmark on synthetic windows, but in full runs its effect ranges from 15% faster (uniform) to 3% slower (clustered data; Section 7.7). A new window assignment replaces the old one only if it is cheaper by more than a relative 10^{-12}. Because the old assignment is feasible for the window problem, every accepted change strictly lowers the global cost by at least that margin. Hence the cost sequence is monotone, the search terminates, and ties (e.g. duplicated points) never cause cycling.

Boundaries. A subgrid of length L_a is cut at positions $\equiv$ shift (mod 6). Both the first block (before the first cut) and the last (after the last cut) are kept as clipped windows. Every lattice position therefore belongs to exactly one window of every phase, on the upper-left and the lower-right sides alike. A test compares the tiling against an independent reference implementation for many sizes, strides, shifts and random masks. Windows with fewer than two usable cells are skipped, because nothing can move in them.

Rectangular grids. Each axis starts from its own stride, and the strides are halved together. The axis that reaches 1 first stays at 1 while the other continues. For a 1000×50 grid the stages are $(256, 16), (128, 8), \ldots, (16, 1), (8, 1), \ldots, (1, 1)$.

Masks, surplus cells and prime counts. Masked positions never enter a window. Empty but usable cells do take part, so occupants can move into them, and the method therefore also decides which cells stay empty. A prime number of points can be placed in two ways. One is an exact-fit grid whose last row is partially usable (a bijection). The other is a full rectangle with a few surplus cells (an injection). We evaluate both (Section 7.6).

4.2 Implementation

The kernel is C++17, exposed to Python with nanobind, and parallelized with OpenMP over the windows of a phase. Each thread keeps its own scratch buffers. Because windows in a phase are disjoint and each solve is deterministic, the result is bit-identical for any thread count; a test checks this. We also track, for every cell, the last phase that changed its occupant. A window whose cells are unchanged since it was last solved in the same stage is already optimal, and skipping it does not change the result (also tested). This matters in the stride-1 convergence rounds, where most windows are untouched. The cost is recomputed exactly after every phase, which gives an anytime trace, and a time budget is checked between windows. There is no just-in-time compilation, so nothing is compiled at run time. Compiling the extension takes 2.6 s with parallel Ninja (7.8 s

on one core). A complete isolated wheel build, including creating the build environment as `pip install` does from source, takes about 35 s.

Work per level. A phase at stride s has about M/w^2 windows (plus clipped ones), each an $O(w^6)$ Hungarian solve in the worst case. The work per phase is therefore $O(Mw^4)$, independent of s, and a schedule with ℓ levels costs $O(\ell M w^4)$ plus the convergence rounds. With $\ell = O(\log L)$ this is $O(M \log M)$ for fixed w. The constant depends strongly on the data: windows far from optimal need long augmenting paths.

4.3 Improvements and non-improvements

We tuned only on a development set (instance seed 0) and report every final claim on held-out seeds. Four changes were tested systematically (Section 7.7):

- **Stride ratio.** Dividing strides by $\sqrt{2}$ instead of 2 (rounded to integers, duplicates removed) roughly doubles the number of levels. Each level costs about the same, but the configuration entering a level is already better. At equal window size this consistently lowered the excess. Ratios of 3 or 1.1 were worse, and so was using every integer stride (development set only).
- **Window size.** Larger windows help at every level. Their cost grows as w^4 in the worst case, but in our runs as about w^2 (uniform) to $w^{2.8}$ (clustered data). Two measured effects contribute: nearly sorted windows solve 2–3× faster than random ones, and unchanged windows are skipped. Using small windows at coarse strides and large ones at stride 1 was worse than using large windows throughout.
- **Start.** Structured starts are *worse* than a random permutation: bisection (the recursive median split used by RasterFairy-like methods), row sorting, and bisection followed by shuffling within $B \times B$ tiles. They create straight seams that the local windows cannot repair. The random start is a decorrelating device, not a lazy default.
- **Solver.** On windows with randomly placed occupants, JV with augmenting row reduction is 1.5× (6×6) to 15× (16×16) slower than the Hungarian method, because squared distances create many near-ties. Only on nearly sorted 6×6 windows is it faster (7 vs. 11 μs). Polishing with extra stride-2 rounds after convergence changed nothing on uniform points and removed under 3% of the excess on clustered data (held-out check; a stride-2 window cannot move a point by one cell). Merging independent runs component by component, the rule BSP-OT uses, helps less per unit time than a larger window.

We keep the baseline exactly as specified and add three presets on the $\sqrt{2}$ schedule: *fast* (6×6), *balanced* (8×8, the package default), and *quality* (12×12).

5 A certified exact solver for large instances

Excess over the optimum is only meaningful with a trustworthy optimum. At $N = 100{,}000$ a dense matrix does not fit in memory. SciPy's sparse LAPJV on a 16-nearest-cell graph did not finish a 10,000-point instance in eight minutes; we later found that such graphs are usually *infeasible*, because some cells are not among any point's 16 nearest. We therefore built a solver that never forms the dense problem, yet certifies optimality against it.

1. **Candidates.** Give each point its 12 nearest cells, plus the 5×5 lattice neighbourhood of its cell in a starting assignment. Including the starting assignment guarantees that a perfect matching exists.
2. **Sparse solve.** Solve the min-cost flow (points $\to$ cells $\to$ sink, unit capacities) with OR-Tools' cost-scaling solver. The solver needs integer costs, so we scale by $S = 2^{56}/((N+M+2)\max c)$ and round. (We found by probing that a scale of $2^{58}/((N+M)\max c)$ is accepted and $2^{60}/((N+M)\max c)$ is rejected with `BAD_COST_RANGE`.)
3. **Potentials.** For the sparse optimum σ, build a graph on cells with an arc $\sigma(i) \to j$ of weight $c_{ij} - c_{i\sigma(i)}$ for every candidate edge. Shortest-path distances from a virtual source joined to all cells give potentials v that satisfy complementary slackness on every candidate edge, with $v = 0$ on empty cells. We compute them in exact integer arithmetic with a label-correcting algorithm.
4. **Global pricing.** Evaluate $\mathrm{LB}(v)$ (Proposition 1) over *all* cells. Cells are grouped into 16×16 blocks, and a block is scanned only if $\mathrm{dist}^2(p_i, \mathrm{block}) - \max_{\mathrm{block}} v$ could beat the best value found so far. The pass also lists every cell whose reduced cost undercuts the point's current cell. If the bound meets the cost, stop. Otherwise add those edges, plus the 3×3 neighbourhood of each such point's best-priced cell, and go back to step 2.

The certificate is the gap $C(\sigma) - \mathrm{LB}(v)$, computed in floating point from real-valued costs. It is valid whatever errors occur in steps 1–3. Integer rounding can only loosen it, by at most $4N/S$ in total; we count an instance as certified when the gap is at most $\max(10^{-9}C, 4N/S)$. On small instances ($N \le 2{,}500$, 720 instances across six distributions and three grid types), the certified optimum agreed with SciPy's dense solver to relative 10^{-9} every time (Section 7.1).

A pitfall. Our first version flagged a "negative cycle" on some clustered instances with masks. It declared a cycle once a node had been queued more than n times, which is only valid for FIFO Bellman–Ford, not with the small-label-first queue we use. The fix is to verify a suspected cycle by walking parent pointers. A regression test covers it.

Cost. Started from ShuffleSnap, the solver reached a certificate on every instance we gave it: the 211 reference instances (up to 200,000 points), every timed run, and uniform instances up to 10^6 points. Started from bisection, it failed on 4 of 11 instances at 10^5 points (Section 7.2). The number of pricing rounds was 1 for uniform inputs, up to about 8 for clustered, curved or masked inputs, and up to 13 for the input with 16 heavily duplicated sites. Massive ties are the slowest case: one such 100,000-point instance took 34 minutes, while the median 100,000-point reference took under 3 minutes (six references were computed in parallel). Section 7 reports its running time and memory next to the heuristics. The solver needs a feasible starting assignment. We use ShuffleSnap's default preset, and at 10^5 points we also run it from a bisection start, to separate the two methods' contributions.

6 Experimental setup

Hardware and software. All runs used one workstation: an Intel Core i9-12900KS (8 performance and 8 efficiency cores, 24 hardware threads), 62 GB RAM, Ubuntu 24.04 (Linux 6.8), Python 3.12.3, NumPy 2.5.3, SciPy 1.18.1, OR-Tools 9.15, GCC 13.3 with `-O3` and no `-march=native` (portable build). The GPU was not used. Exact versions and commit hashes are stored with every

raw result. *Single-thread* runs were pinned to one performance core (`taskset -c 8`) with all thread pools set to 1. *Multi-thread* runs used all 24 hardware threads without pinning. For ShuffleSnap this was faster on this machine than using only the 16 performance-core threads (Section 7.9). The machine also ran unrelated background services (load average about 1). Benchmarks ran sequentially, only after all reference optima had been computed.

Timing and memory boundaries. Every run is a fresh process. The timer brackets exactly one call of the method adapter. It includes all method-specific work: cost matrices, trees, starts, the method's own normalization, and format conversion. It excludes instance generation, the common normalization, validation, and metrics. Before the timed call, each method is run once on a 256-point instance. This takes imports, lazy initialization and thread-pool start-up out of the timing; that warm-up time is recorded separately and is below 0.3 s for every method. None of the methods uses just-in-time compilation. ShuffleSnap's one-time C++ build is not part of any run: 2.6 s to compile, or about 35 s for an isolated wheel build. *Peak memory* is the rise of the process's peak resident set size during the call: the kernel counter is reset through `/proc/self/clear_refs` just before the call, and the resident size at that moment is subtracted. It therefore measures what the method needs on top of its inputs.

Seeds and held-out evaluation. All design decisions were made on instance seed 0. Reported results use instance seeds 1–3, which were not used for tuning. For the real embeddings, which have one fixed instance each, the seeds vary only the methods' own randomness.

Data. Synthetic distributions: uniform square, isotropic Gaussian, a 10-component anisotropic Gaussian mixture (*gmm*, cluster structure like typical embeddings), two clusters with 90/10 mass (*unequal2*), a thin ring, a noisy spiral (a 1D manifold), a checkerboard density with a 4:1 contrast, Student-t with 1.5 degrees of freedom (*heavy*), 16 locations with many duplicates, a nearly degenerate line, and the target lattice itself with Gaussian jitter ($\sigma = 0.25$ cells), whose optimum is nearly the identity. Real data: UMAP embeddings [30] of all 70,000 MNIST digits [24] and all 70,000 Fashion-MNIST images [48] (15 neighbours, min_dist 0.1, fixed seed). Targets: an exact-fit near-square grid (*square*, last row partially usable when needed), the full rectangle with its few surplus cells (*free*), a rectangle with 10% surplus cells (*free10*), a 4:1 grid (*wide*), a disk, a ring, and a mask shaped like the letter S (*logo*).

Comparators and adapters. Each adapter receives the same normalized float64 coordinates and cell centers and returns one cell per point. We check validity (injective, usable cells only) independently of every method.

- *Exact, dense*: SciPy `linear_sum_assignment` on the float64 cost matrix, the fastest of three dense solvers we tried (`lap.lapjv` and `lapjv` were 3–8× slower at $N = 2{,}500$).
- *Exact, certified sparse*: Section 5, started from ShuffleSnap's default (balanced) preset, whose time is included, or from bisection.
- *RasterFairy* [21]. The PyPI release 1.0.6 (2019) uses the removed NumPy alias `np.float`, and it orders points by *integer-truncated* coordinates. With grid-unit inputs this discards sub-cell order: excess was 193% without our remedy of multiplying inputs by 1000, and 82% with it (uniform, $N = 2{,}500$). RasterFairy's optional swap optimizer is excluded: the PyPI copy is Python 2 code (a `SyntaxError` on Python 3), and the upstream copy, which does run on Python 3, allocates a dense $N \times N$ distance matrix (80 GB at 10^5 points). The upstream

GitHub master (July 2026) normalizes internally, sorts on floats, and solves leaf blocks of up to 200 points exactly with SciPy. It is much better (9% excess on the same instance), and it is our representative setting. Its per-leaf console printing is captured so that it does not inflate the timing. Surplus cells are removed at random by its `autoAdjustCount` (default strategy).

- *BSP-OT* [12] through its official Python binding `bspot` 1.2.0 [43]. Bijections use its matching function with Gaussian slicing, as in the upstream demo; surplus cells use its partial-matching function, which has no Gaussian option. We report 16 plans (the default) and 64 plans (as in the paper's figure), and sweep 1–1024 plans for time–quality curves. The binding always runs the cycle-merge step, whereas the upstream C++ wrapper disables it at $N \geq 500{,}000$.
- *Bisection*: the recursive median split alone, an $O(N \log N)$ baseline close to one BSP matching with axis-aligned cuts.

Neighbourhood measures. Displacement is not neighbourhood preservation, so we measure the latter separately on 5,000 sampled points: *kNN recall*, the fraction of each point's 10 nearest input neighbours that are among its 10 nearest grid neighbours (ties at the 10th grid distance count), and *lattice precision*, the fraction of occupied 8-neighbour cells whose points are among the 8 nearest input neighbours.

7 Results

7.1 Correctness checks

Three properties were checked independently of the kernel:

- **Window solver.** Two unit tests cover it. The first compares against brute-force enumeration on random, tie-heavy and badly scaled matrices up to 5×6. The second compares against SciPy's `linear_sum_assignment` on square and rectangular matrices up to 144×144, including squared-distance matrices like the real use case. Both run for all three window solvers.
- **Validity.** A NumPy validator, sharing no code with the kernel, checked that every output of every method gives each point its own usable cell. All 943 completed runs of the main benchmark were valid.
- **Monotonicity.** Each ShuffleSnap run records the exact cost after every phase. In all 396 ShuffleSnap runs of the main benchmark, this trace was monotone, and its final value matched the cost recomputed by the validator.

The test suite also checks that results do not depend on the thread count or on skipping unchanged windows, and that the window tiling matches an independent reference implementation.

Small instances. On 720 small instances ($N \in \{100, 400, 900, 2500\}$; six distributions; exact-fit, 10%-surplus and disk targets; 10 seeds each), the certified solver agreed with SciPy's dense solver on every instance. Table 1 shows how often each method reached the optimum. ShuffleSnap is reliably exact only when its windows cover a large part of the grid: the quality preset's 12×12 window covers the whole 10×10 grid at $N = 100$. By $N = 2{,}500$, the baseline never reached the optimum, although its median excess was only 0.18%.

Table 1: Small instances: fraction of the 180 instances per size (six distributions, three target types, 10 seeds) on which the method reached the certified optimum (relative excess $< 10^{-9}$), and the median excess.

	SS base	SS fast	SS bal.	SS qual.	BSP-OT 16	BSP-OT 64
$N = 100$	0.57 / 0.000%	0.57 / 0.000%	0.86 / 0.000%	1.00 / 0.000%	0.04 / 9.3%	0.08 / 4.2%
$N = 400$	0.11 / 0.195%	0.12 / 0.157%	0.30 / 0.029%	0.58 / 0.000%	0.00 / 11.9%	0.01 / 5.6%
$N = 900$	0.03 / 0.204%	0.03 / 0.152%	0.09 / 0.062%	0.23 / 0.013%	0.00 / 11.6%	0.00 / 6.2%
$N = 2500$	0.00 / 0.180%	0.01 / 0.099%	0.03 / 0.045%	0.08 / 0.011%	0.00 / 12.3%	0.00 / 6.4%

Table 2: Mean excess over the certified optimum (%) at $N = 100{,}000$, square exact-fit grid, instance seeds 1–3, one thread. The last two columns give the time of the certified exact solver (seed 1) started from ShuffleSnap-balanced (time included) or from bisection. ^{t}: no certificate within the 1-hour limit; ^{n}: stopped after 50 pricing rounds without a certificate. The bottom row is the median time over all runs. Heavy tails are below 0.001% for every method in relative terms; Table 7 gives absolute excess.

distribution	SS base	SS fast	SS bal.	SS qual.	BSP 16	BSP 64	RF master	RF PyPI	Bisect.	exact (s)	exact, bis. (s)
uniform	2.51	1.50	0.84	0.44	44.8	32.1	44.8	61.6	100.3	16.9	67.9
Gaussian	<0.001	<0.001	<0.001	<0.001	0.055	0.040	0.006	0.010	0.018	71.5	141
Gaussian mixture	0.011	0.008	0.002	<0.001	5.61	3.74	16.7	16.7	12.3	110	>3600^{t}
90/10 clusters	0.031	0.034	0.012	<0.001	0.032	0.019	1.16	1.16	0.57	230	1048
ring	0.018	0.003	0.001	<0.001	1.16	0.95	41.0	41.0	40.3	128	3117
spiral	0.023	0.012	0.004	0.001	9.96	8.06	27.3	27.4	34.4	129	3331^{n}
checkerboard	0.17	0.091	0.057	0.020	49.8	36.1	425.2	426.3	450.0	68.7	423
heavy tails	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	277	320
duplicates	0.10	0.086	0.038	0.013	12.3	5.44	49.0	49.0	39.3	1384	>3600^{t}
near-line	0.010	0.008	0.002	<0.001	0.008	0.002	19.4	19.4	19.1	273	>3600^{t}
jittered lattice	32.8	22.9	12.7	7.41	64.9	45.9	13.2	29.3	282.7	7.09	5.23
median time (s)	5.07	6.89	15.9	52.4	0.72	2.85	2.99	81.8	0.034	128	

7.2 Quality and time at 10^5 points

Table 2 and Figure 2 summarize the held-out comparison at 10^5 points on one thread.

ShuffleSnap. On nine of the eleven distributions, the baseline as specified is within 0.2% of the optimum, and on six of them within 0.03%. It needs a median of 5 s on one thread. The two exceptions are uniform points (2.5%) and the jittered lattice (33%). There the optimum is small, so the long loops of Section 7.8 dominate. The presets trade time for quality almost monotonically. The one exception is the 90/10 clusters, where fast is slightly worse than the baseline (0.034% vs. 0.031%). Balanced (median 16 s) reduces the uniform excess to 0.84%, and quality (median 52 s) to 0.44%. The jittered lattice remains hard even for quality (7.4%): in absolute terms, 0.03 cells^2 per point against an optimum of 0.43 (Table 7).

BSP-OT is the fastest method (0.7 s for 16 plans, 2.9 s for 64). Its excess, however, is typically 10–350× that of the ShuffleSnap baseline: 32% on uniform points, 36% on the checkerboard, 8% on the spiral and 3.7% on the mixture, all with 64 plans. It is competitive on two inputs. On the near-line, 64 plans (0.002%) match the balanced preset, and on the 90/10 clusters they beat the baseline (0.019% vs. 0.031%) but not balanced (0.012%).

RasterFairy (upstream) takes about 3 s. Its recursive slicing, however, costs 17–425% on seven of the eleven distributions. It does well on three: the Gaussian (0.006%), the 90/10 clusters (1.2%), and the jittered lattice, where it matches the balanced preset (13.2% vs. 12.7%) because slicing a noisy lattice recovers the lattice. The PyPI release, with pre-scaled inputs, gives similar or worse

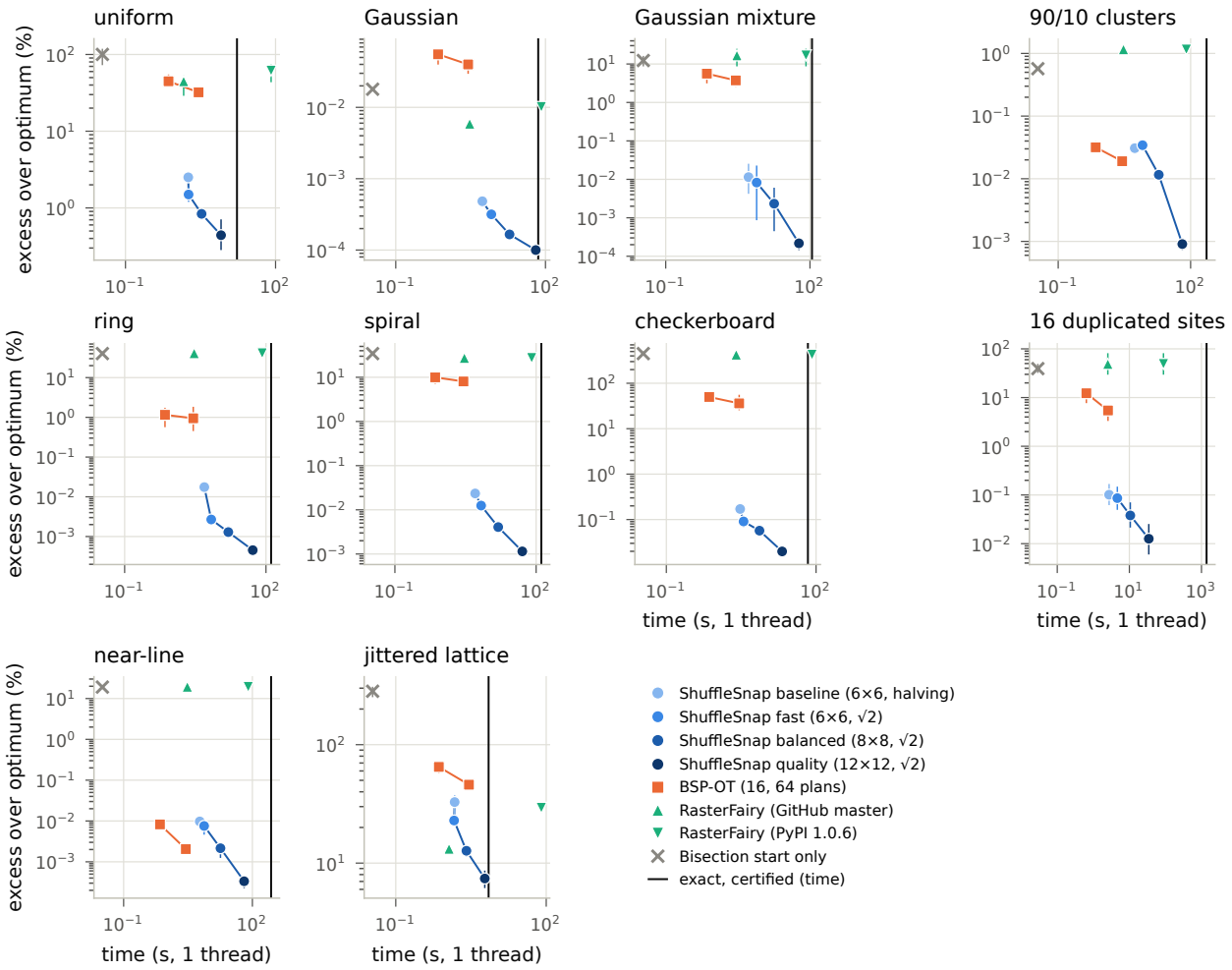


Figure 2: Excess over the certified optimum against single-thread time at $N = 100{,}000$ (mean over three held-out instances; bars span min–max). The four ShuffleSnap presets form the blue curve, from baseline to quality. BSP-OT at 16 and 64 plans is joined by an orange line. The black vertical line is the time of the certified exact solver. Heavy tails are omitted (all methods <0.001%).

costs (e.g. 62% vs. 45% on uniform points) and takes 80 s at this size. Its targets at 50k and 100k points have a partially filled last row, which sends it into its much slower mask code path; at $N = 10^4$, a full 100×100 square, it needs 0.03 s.

With all cores. On 24 hardware threads, ShuffleSnap ran 13–15× faster than on one and returned bit-identical assignments in all 396 paired runs. At 10^5 points the median time is 0.35 s for the baseline, 1.1 s for balanced and 3.9 s for quality. BSP-OT gained 4–7× (0.13 s with 16 plans, 0.42 s with 64); its plans are computed in parallel, but the cycle-merging pass that combines them is largely sequential. At 24 threads, the baseline and BSP-OT with 64 plans therefore take about the same time, and the baseline has lower excess on nine of the eleven distributions. The two exceptions are the near-line and the 90/10 clusters, as above.

Exact solutions. Started from ShuffleSnap, the certified solver finished every instance, in 7 s (jittered lattice) to 23 min (16 duplicated sites), with a median of 128 s. That is 5–500× the time of the ShuffleSnap baseline (median 25×). Started from bisection instead, it failed on four instances

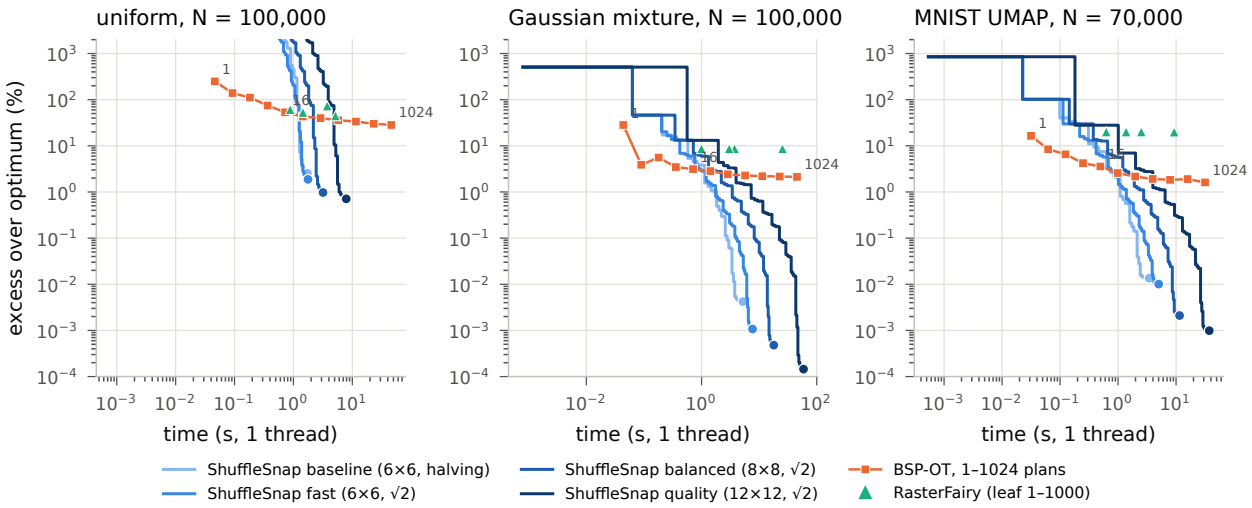


Figure 3: Excess over the certified optimum against elapsed time, one thread. ShuffleSnap's step curves record the exact cost after every phase, so each shows the result that a time budget ending at that moment would return. BSP-OT is run separately with 1, 2, 4, …, 1024 plans (labels at 1, 16 and 1024). RasterFairy upstream is shown with exact leaf blocks of 1, 50, 200 and 1000 points. Instance seed 1. The same plot with 24 threads is in the appendix (Figure 10).

(mixture, duplicates, near-line, spiral): no certificate within an hour or 50 pricing rounds. On six of the seven instances where it did certify, it was 1.2–24× slower; on the jittered lattice it was slightly faster (5 s vs. 7 s). The start matters because the candidate edges come from it: ShuffleSnap's layout is close enough to optimal that few pricing rounds are needed. A dense exact solver is not an option at this size. Already at $N = 10^4$, SciPy needed 3–352 s (Table 8 in the appendix), whereas the certified sparse solver needed 0.3–7.7 s.

7.3 Quality at a matched time budget

Figure 3 compares the methods at matched time budgets. BSP-OT improves slowly with more plans and levels off far from the optimum. From 1 to 16 to 1024 plans, its excess drops from 249% to 53% to 28% on uniform points, from 28% to 3.1% to 2.1% on the mixture, and from 16% to 3.6% to 1.6% on MNIST. The 1024-plan runs take 31–46 s on one thread. The ShuffleSnap baseline passes that best BSP-OT result after 1.1–1.3 s and finishes at 2.5%, 0.004% and 0.014% respectively. At equal elapsed time, ShuffleSnap is ahead from 0.7–1.3 s onward. With less time than that (on one thread at 10^5 points), BSP-OT is the better choice, because ShuffleSnap has only finished its coarsest levels. With 24 threads, the crossover moves to 0.04–0.09 s. RasterFairy's leaf size hardly matters except on uniform points (73% with no exact leaves, 45% with 1000-point leaves).

7.4 Scaling in time and memory

Between 10^4 and 10^6 points, ShuffleSnap, BSP-OT and RasterFairy all scale close to linearly. Their fitted exponents of time in N are 1.11–1.18, 1.08 and about 1.0 (Figure 4). On one thread, 10^6 uniform points take 25 s for the baseline and 45 s for balanced; with 24 threads, 1.6 s and 2.9 s. On the mixture, 10^6 points take 68 s and 248 s on one thread, or 4.4 s and 17 s on 24 threads. BSP-OT (16 plans) and RasterFairy take 9 s and 13–20 s on one thread.

Exact solving scales worse but reaches further than we expected. The certified solver's time grew as $N^{1.5}$ on uniform points and $N^{1.6}$ on the mixture. It certified 10^6 uniform points in 11.8 min

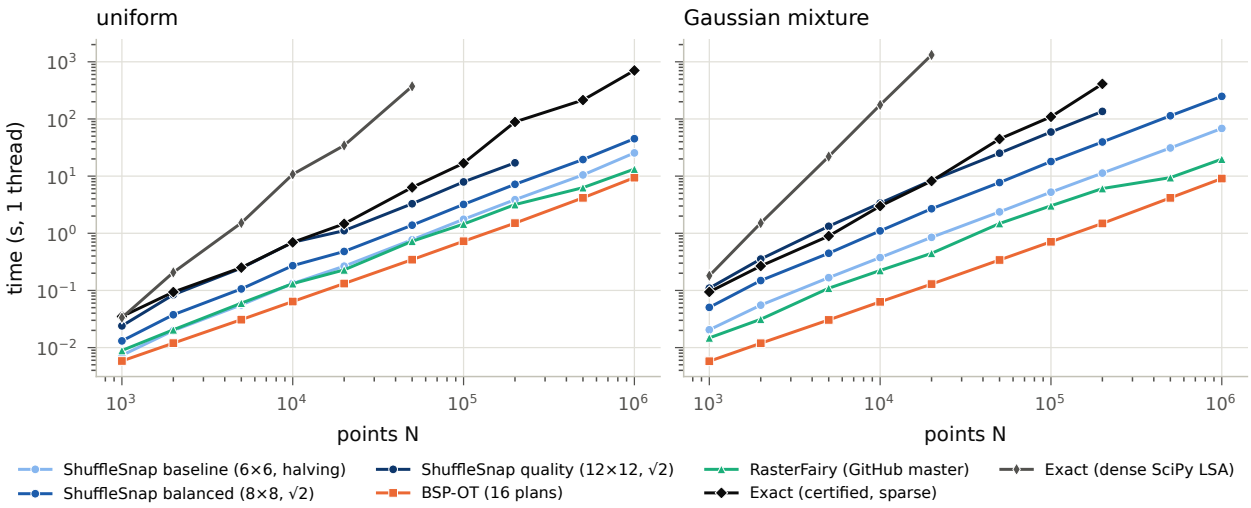


Figure 4: Time against N on one thread (instance seed 1). Dense SciPy was run up to the largest size we considered sensible, 50k uniform points (6.2 min, 23 GB) and 20k mixture points (22 min). The certified solver was run up to 10^6 uniform and 2×10^5 mixture points. The quality preset was run up to 2×10^5 points.

Table 3: Peak extra resident memory (MB) during one call, measured on a heap trimmed right before the call (`malloc_trim`), one thread, instance seed 1. Without trimming, small allocations can reuse pages freed while the instance was generated and go unseen.

	uniform			Gaussian mixture		
method	10^4	10^5	10^6	10^4	10^5	10^6
ShuffleSnap baseline	0.4	2.9	36.7	0.4	3.0	37.1
ShuffleSnap balanced	0.3	2.9	22.2	0.3	3.0	22.9
BSP-OT, 16 plans	2.2	24.2	243	2.4	24.3	244
RasterFairy master	6.2	63.4	647	6.2	62.9	647
exact, certified sparse	49.4	421	–	81.1	767	–
exact, dense SciPy	1563	–	–	1564	–	–
bisection only	0.1	1.2	11.4	0.1	1.1	13.3

using 4.3 GB, and 2×10^5 mixture points in 6.8 min. Its memory, 4–8 kB per point, is set by the candidate edges and the flow solver. Dense SciPy grew as $N^{2.2}$ (uniform) to $N^{2.9}$ (mixture) and needs $8N^2$ bytes. It took 6.2 min and 23 GB at 50k uniform points, and 22 min at 20k mixture points. ShuffleSnap needs $O(N)$ working memory: an occupancy array, a position array, a last-change stamp per cell, and the window lists of the current level. Measured, this is 22–37 bytes per point, or 37 MB at 10^6 points (Table 3). The comparators need 7–29× more: BSP-OT about 240 and RasterFairy about 650 bytes per point.

At 10^6 uniform points, the quality ranking is unchanged. Measured against the certified optimum of that instance, the excess is 5.3% for the baseline, 2.0% for fast and 1.3% for balanced, against 74% for BSP-OT (16 plans) and 48% for RasterFairy. The baseline's excess keeps growing with N (1.1% at 10^3, 2.5% at 10^5 and 5.3% at 10^6, one instance each); the finer schedules grow much more slowly (balanced: 0.1%, 1.0%, 1.3%).

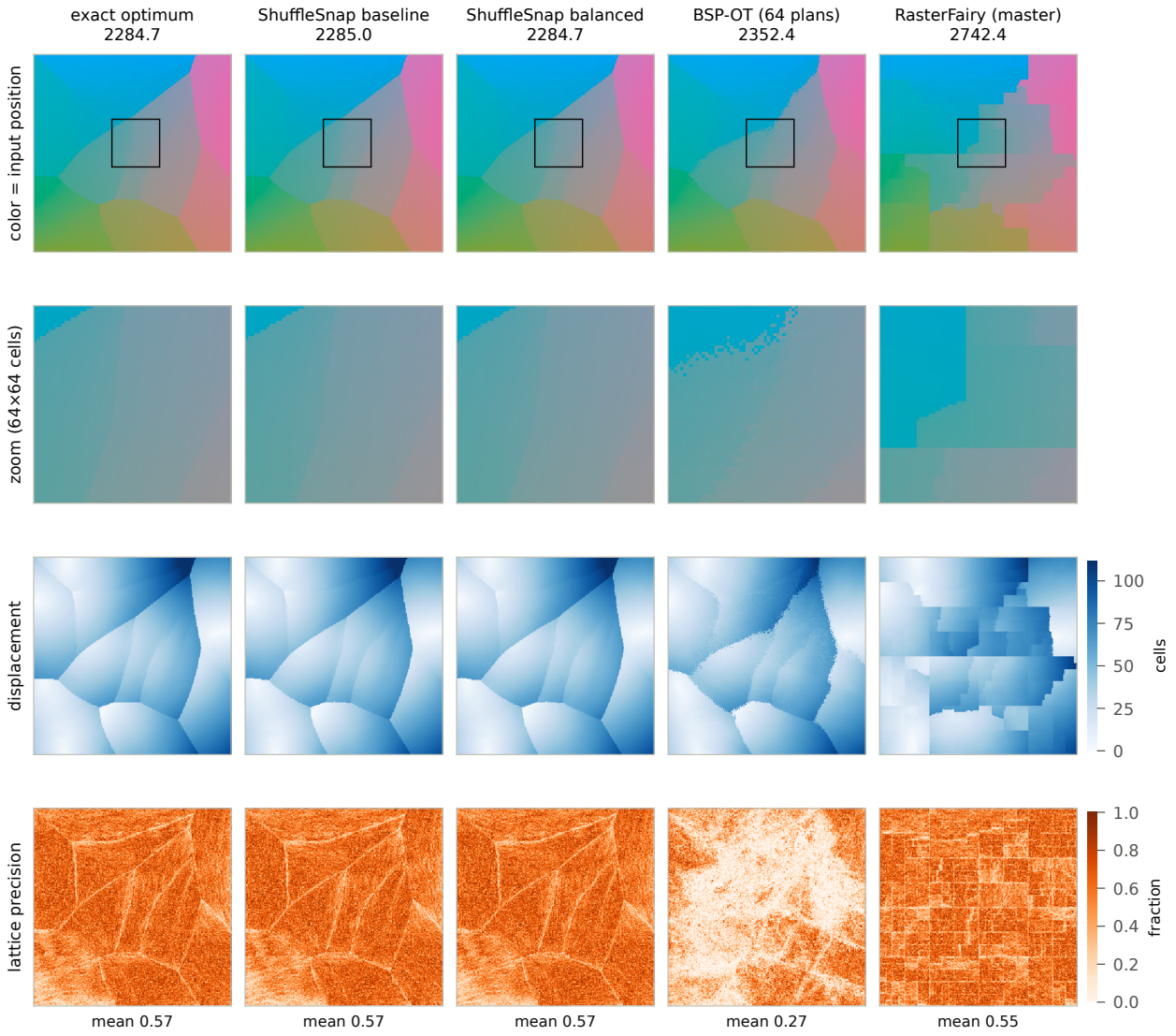


Figure 5: All 70,000 MNIST digits, UMAP layout snapped to a 265×265 grid. Each point has the same color in every panel, set by its position in the input layout, so color jumps show where a method tears the layout. The number under each method name is the mean squared displacement in cells^2. Row 2 enlarges the 64×64 box. Row 3 shows each point's displacement. Row 4 shows lattice precision: the fraction of a point's occupied 8-neighbour cells whose points are among its 8 nearest input neighbours (higher is better). ShuffleSnap is visually indistinguishable from the certified optimum. BSP-OT is within 3% of the optimal displacement but halves lattice precision, because it scrambles local order (speckled boundaries in row 2). RasterFairy's recursive slicing leaves rectangular seams.

Table 4: Real embeddings, all 70,000 points (square exact-fit grid). Excess over the certified optimum and neighbourhood measures are means over three method seeds; times are medians on one thread (1T) and on 24 threads (24T, methods that use threads only). Memory is the rise of peak resident memory during the call. kNN recall: fraction of each point's 10 nearest input neighbours among its 10 nearest grid neighbours. Lattice precision: fraction of occupied 8-neighbour cells holding one of the point's 8 nearest input neighbours.

	method	excess (%)	mean sq.	time 1T (s)	time 24T (s)	mem (MB)	kNN recall	lattice prec.
MNIST UMAP	exact (certified)	0	2284.7	85.2	–	568	0.645	0.581
	ShuffleSnap baseline	0.015	2285.0	3.45	0.24	2	0.644	0.578
	ShuffleSnap fast	0.010	2284.9	5.01	0.34	2	0.643	0.578
	ShuffleSnap balanced	0.002	2284.7	11.5	0.80	2	0.645	0.580
	ShuffleSnap quality	<0.001	2284.7	37.4	2.80	2	0.644	0.580
	BSP-OT, 16 plans	3.73	2370.0	0.49	0.088	16	0.305	0.255
	BSP-OT, 64 plans	2.42	2340.0	1.98	0.28	42	0.305	0.249
	RasterFairy master	20.0	2742.4	1.38	–	42	0.631	0.551
	RasterFairy PyPI	44.5	3301.0	11.2	–	44	0.602	0.534
	bisection only	34.4	3071.1	0.023	–	0	0.569	0.498
Fashion UMAP	exact (certified)	0	1802.6	77.6	–	576	0.606	0.548
	ShuffleSnap baseline	0.009	1802.7	3.37	0.24	2	0.606	0.547
	ShuffleSnap fast	0.003	1802.6	4.73	0.32	2	0.606	0.547
	ShuffleSnap balanced	0.001	1802.6	10.8	0.76	2	0.607	0.547
	ShuffleSnap quality	<0.001	1802.6	35.8	2.69	2	0.607	0.547
	BSP-OT, 16 plans	9.15	1967.5	0.50	0.088	16	0.295	0.245
	BSP-OT, 64 plans	6.73	1923.8	1.98	0.28	42	0.334	0.275
	RasterFairy master	24.6	2245.3	1.38	–	42	0.611	0.532
	RasterFairy PyPI	25.6	2264.5	11.1	–	44	0.594	0.526
	bisection only	23.0	2217.7	0.023	–	0	0.544	0.477

7.5 Real embeddings: displacement versus neighbourhoods

Figure 5 and Table 4 compare the methods on the full MNIST and Fashion-MNIST embeddings. In the runs shown, the balanced preset is within 0.002% of the certified optimum on both datasets, and the baseline within 0.014%. In every panel their colour fields match the optimum's.

The two measures disagree on BSP-OT. With 64 plans, its displacement is only 2.4% (MNIST) and 6.7% (Fashion) above optimal, yet both neighbourhood measures fall to about half: kNN recall 0.31–0.33 against 0.61–0.65 for the optimum, and lattice precision 0.25–0.28 against 0.55–0.58. Averaged over plans, the merged BSP partitions are close to the transport map, but they shuffle points locally. RasterFairy shows the opposite pattern: its displacement is 20–25% above optimal, yet its neighbourhood measures are close to the optimum's. On Fashion-MNIST its kNN recall (0.611) even exceeds the optimum's (0.606), because local order survives inside each slice and breaks only along the seams. ShuffleSnap's measures match the optimum's to within 0.003. Even the optimum has lattice precision well below 1: an optimal transport map from clustered data to a uniform grid tears the layout along cluster boundaries (the white lines in row 4). Minimizing displacement and preserving neighbourhoods are different objectives, and neither implies the other.

7.6 Masks, surplus cells and prime counts

Table 5 and Figure 6 cover non-square targets.

- **Convex masks.** On disks and rings, every ShuffleSnap preset stays within 0.25% (baseline) and 0.07% (balanced) of the optimum. BSP-OT is at 2.5–9% and RasterFairy at 11–54%.
- **The S-shaped mask.** This is the baseline's worst case. With uniform points it is 18.3% above optimal, worse than BSP-OT with 64 plans (13.2%); with the spiral it is 7.3%. The mechanism is the one in Figure 8: coarse strides route mass across the bays. The $\sqrt{2}$ schedule repairs most of this. Balanced reaches 1.3% and 0.26%, and quality 0.13% and 0.04%.

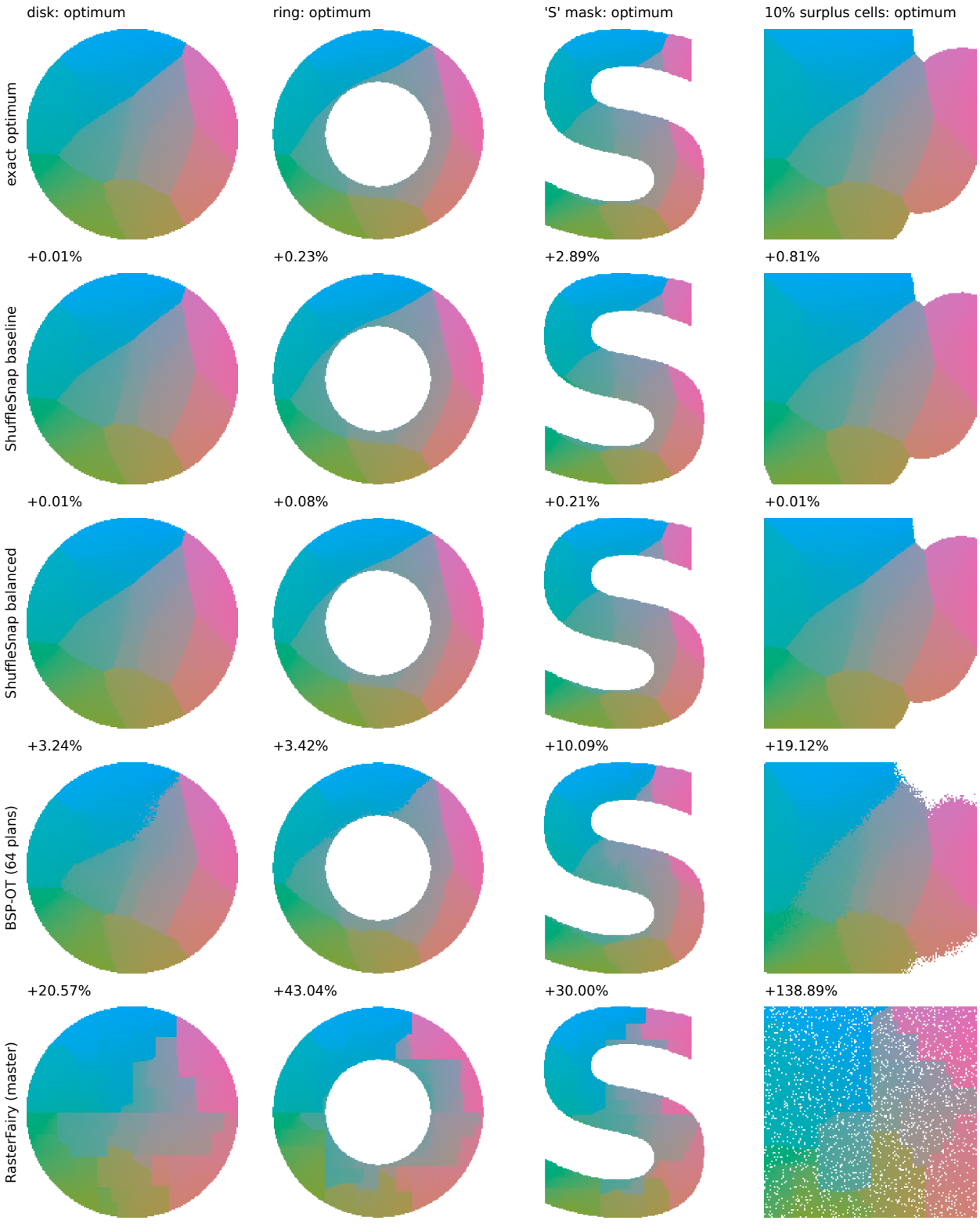


Figure 6: A 20,000-point subset of the MNIST UMAP embedding on a disk, a ring, an S-shaped mask, and a rectangle with 10% surplus cells (instance seed 1; colors encode input position as in Figure 5; white = empty or masked). Numbers are excess over the certified optimum. With surplus cells, the optimum leaves one contiguous region empty, far from the data, and ShuffleSnap-balanced finds the same region. BSP-OT's partial matching leaves ragged holes along the boundary, and RasterFairy removes surplus cells at random before slicing.

Table 5: Excess over the certified optimum (%) on non-square targets, $N = 20{,}000$, mean of instance seeds 1–3 (all 24 exact runs certified). "Surplus (<1 row)" is the full $W \times H$ rectangle holding N points, with fewer than W empty cells; "10% surplus" has $1.1N$ cells.

target	points	SS base	SS fast	SS bal.	SS qual.	BSP 16	BSP 64	RF master
surplus (<1 row)	uniform	8.37	4.52	3.10	1.37	90.3	85.0	56.6
surplus (<1 row)	Gaussian mixture	0.051	0.027	0.013	0.010	8.41	7.95	17.2
surplus (<1 row)	spiral	0.28	0.043	0.006	0.003	19.9	11.5	32.4
surplus (<1 row)	MNIST UMAP	0.11	0.042	0.020	0.011	16.5	5.29	43.6
10% surplus	uniform	4.36	2.22	0.91	0.25	108.9	58.7	350.5
10% surplus	Gaussian mixture	0.95	0.21	0.055	0.012	30.5	17.6	71.6
10% surplus	spiral	2.77	0.21	0.071	0.012	26.2	19.3	157.9
10% surplus	MNIST UMAP	0.70	0.024	0.007	0.003	29.4	22.8	146.2
4:1 rectangle	uniform	2.48	1.67	1.34	0.45	32.3	23.9	26.4
4:1 rectangle	Gaussian mixture	0.11	0.085	0.024	0.004	1.44	0.81	6.70
4:1 rectangle	spiral	0.22	0.16	0.067	0.022	4.33	2.34	15.4
4:1 rectangle	MNIST UMAP	0.10	0.099	0.014	0.003	2.07	1.28	11.6
disk	uniform	0.057	0.023	0.014	0.006	8.64	6.74	11.4
disk	Gaussian mixture	0.009	0.009	0.003	<0.001	5.43	4.27	37.0
disk	spiral	0.029	0.018	0.007	0.003	5.19	2.48	23.6
disk	MNIST UMAP	0.018	0.011	0.006	0.002	4.49	2.77	20.3
ring	uniform	0.084	0.027	0.028	0.006	9.28	6.73	53.6
ring	Gaussian mixture	0.18	0.083	0.048	0.018	6.75	4.81	35.9
ring	spiral	0.15	0.021	0.014	0.016	7.73	3.83	47.9
ring	MNIST UMAP	0.22	0.12	0.070	0.025	4.77	3.11	43.2
S mask	uniform	18.3	1.59	1.30	0.13	16.9	13.2	97.4
S mask	Gaussian mixture	0.95	0.094	0.058	0.008	4.38	3.48	27.9
S mask	spiral	7.33	0.47	0.26	0.043	12.3	6.65	76.4
S mask	MNIST UMAP	3.17	0.28	0.22	0.018	11.8	9.83	30.2

- **Surplus cells.** When the solver must also choose the empty cells, the baseline degrades noticeably: 0.7–4.4% with 10% surplus, and 8.4% for uniform points with less than one row of surplus. Balanced stays below 1% everywhere except that last case (3.1%), and quality stays below 1.4%. Surplus is where the comparators suffer most. With 10% surplus, BSP-OT's partial matching is 18–109% above optimal, and RasterFairy, which picks the empty cells at random, is 72–350% above optimal. With less than one row of surplus, they are at 5–90% and 17–57%.

Prime counts. A prime number of points never fills a rectangle. Table 6 compares the two ways of handling this. With an exact-fit grid, the results match the square case: 2.1–3.0% for the baseline on uniform points, and about 0.01% on the mixture. Leaving the solver to pick fewer than 200 empty cells in a full rectangle is much harder on uniform points. The baseline reaches 12.6–14.5%, balanced 5.1–5.7% and quality 3.0–3.6%; for the baseline this is 0.15–0.16 cells^2 per point, about four times the exact-fit case. BSP-OT's partial matching becomes worse still, at 147–152%. For layouts where the empty cells' position does not matter, an exact-fit mask is therefore both simpler and better handled by every method.

Table 6: Prime point counts, excess over the certified optimum (%), mean of instance seeds 1–3 (24 threads; all 8 timed exact runs certified). The *exact fit* target is a near-square grid whose last row is partially usable, so every cell is filled. The *full rectangle* has a few surplus cells (177 for $N = 49{,}999$, 181 for $N = 99{,}991$), and the solver must also choose which cells stay empty.

N	target	points	SS base	SS bal.	BSP 16	RF master
49,999	exact fit	uniform	2.11	0.85	36.8	31.7
49,999	exact fit	Gaussian mixture	0.012	0.002	5.32	36.9
49,999	full rectangle	uniform	14.5	5.72	146.6	81.9
49,999	full rectangle	Gaussian mixture	0.19	0.079	12.0	39.1
99,991	exact fit	uniform	3.03	1.19	56.1	47.4
99,991	exact fit	Gaussian mixture	0.013	0.003	4.77	16.0
99,991	full rectangle	uniform	12.6	5.14	152.4	70.7
99,991	full rectangle	Gaussian mixture	0.100	0.021	8.21	17.1

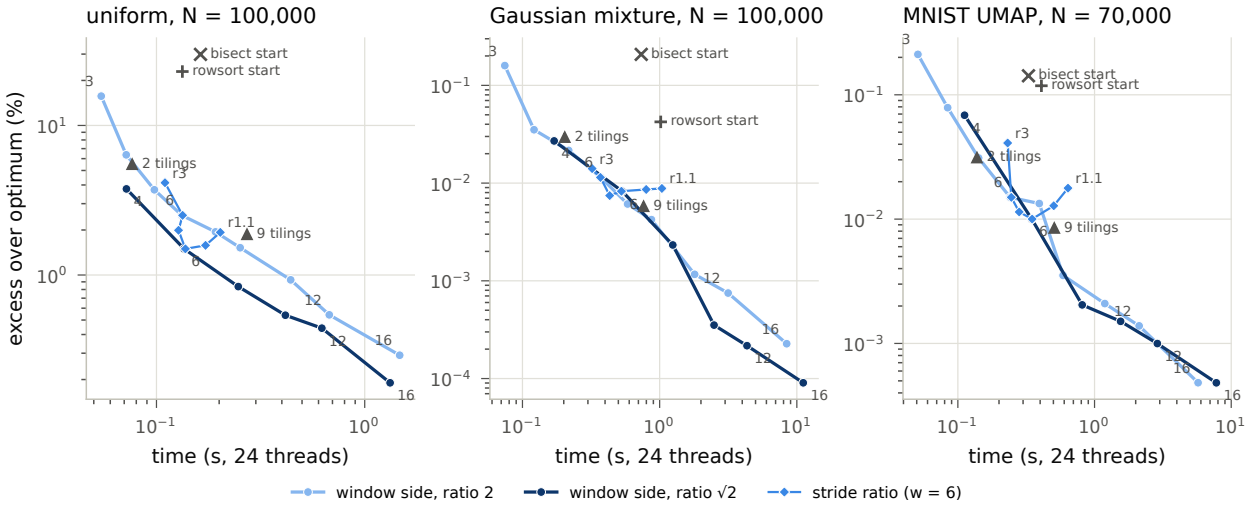


Figure 7: Held-out ablations. Each panel shows mean excess over the certified optimum against median time (24 threads, three instance seeds). Lines vary an ordered setting of the baseline: the window side, with halving (light) or $\sqrt{2}$ strides (dark), labelled by the side length; and the stride ratio at $w = 6$ (diamonds; r3 and r1.1 mark the extremes). Markers show categorical changes to the baseline: bisection or row-sort starts, and 2 or 9 shifted tilings per round instead of 4. A single tiling (>20,000% on uniform points) is off the chart. Solver choice and skipping unchanged windows do not change quality; they are listed with all other numbers in Table 10.

7.7 Ablations on held-out instances

Figure 7 varies one design choice at a time, starting from the baseline, on instances that were not used for any design decision.

- **Window side** is the main quality knob. With halving strides, uniform excess falls from 15.7% (3×3) to 2.5% (6×6), 0.54% (12×12) and 0.29% (16×16). Time rises from 0.05 s to 1.5 s on uniform points and from 0.07 s to 8.4 s on the mixture. The $\sqrt{2}$ schedule shifts the whole curve down (e.g. 1.5% at 6×6).
- **Stride ratio**. $\sqrt{2}$ is best or close to best on all three inputs: on uniform points 1.5% against 2.5% for halving and 4.1% for a ratio of 3, and 1.9% for the finest ratio tested, 1.1. Finer is not always better.

- **Coarse levels are essential**. Capping the first stride at 16, 8, 4, 2 or 1 (the last means stride-1 windows only, from the random start) made results 5–37× worse and 5–90× slower. With halving from the automatic first stride of 64, uniform excess is 2.5% in 0.12 s; capped at 16 it is 11.7% in 0.57 s, and with stride 1 only it is 27% in 1.3 s. Without long-range moves, stride-1 windows need many rounds to repair a random start, and they stall far from the optimum.
- **Start**. A random start beats bisection and row sorting by 4–18× (uniform: 2.5% against 29.9% and 22.9%). Better-looking starts contain straight seams that windows cannot undo.
- **Shifted tilings are essential**. With a single fixed tiling, points can never cross a window border, and the method fails (uniform: excess over 20,000%). Two diagonal shifts give 5.5%, the four of the baseline 2.5%, and nine shifts 1.9% at twice the time. More rounds per level help about as much per second as larger windows (two rounds: 2.1% at 1.4× the time).
- **Window solver**. All three solvers are exact, so the assignments are identical. Jonker–Volgenant is 2–10× slower than the Hungarian method. The greedy warm start saves up to 15% on uniform inputs but is 2–3% slower on the clustered ones, so its benefit in the micro-benchmark (5–20%) does not carry over consistently.
- **Skipping unchanged windows** leaves results unchanged and saves 35–40% of the time.
- **Post-processing does not pay.** Three extra cycles of stride-2 and stride-1 rounds after convergence changed nothing on uniform points and removed under 3% of the excess on the clustered inputs. Merging four independent baseline runs, keeping the cheaper of the two assignments on every connected component of their union (the rule BSP-OT uses to merge plans), cut the uniform excess by 23–38% at four times the cost. The fast preset does better (1.5%) at about the same cost as the baseline on these uniform instances.

7.8 Anatomy of the remaining error

Why does a method that solves every window exactly still miss the optimum by a few percent on uniform data? Compare a ShuffleSnap layout with the certified optimum. The permutation that turns one into the other splits into cycles (Figure 8a). Almost all the excess sits in a handful of very long cycles, in which every point is shifted by about one cell, so the layout is locally rotated along a closed curve (Figure 8b). Such a loop is invisible to every window:

- Inside a stride-1 window, the part of a loop is a run of points shifted consistently by one cell, which is already the best arrangement of that occupant set. Undoing the loop requires moving one point out at one end of the run and one in at the other end.
- Coarser windows cannot help either. A one-cell shift moves a point to another residue class, and a strided window only permutes points within one class.

Indeed, three extra cycles of stride-2 and stride-1 rounds after convergence changed nothing on held-out uniform instances, and removed under 3% of the excess on clustered ones.

A constructed fixed point. The argument can be checked directly. Put the points exactly on the cell centers of a 64×48 grid, so that the optimum is the identity with cost 0. Then rotate an axis-aligned ring of cells by one step: the point of each ring cell moves to the next ring cell, so the cost equals the number L of ring cells. We ran the complete $\sqrt{2}$ schedule from this layout, with every stride and every tiling, followed by stride-1 rounds to convergence. For windows of side

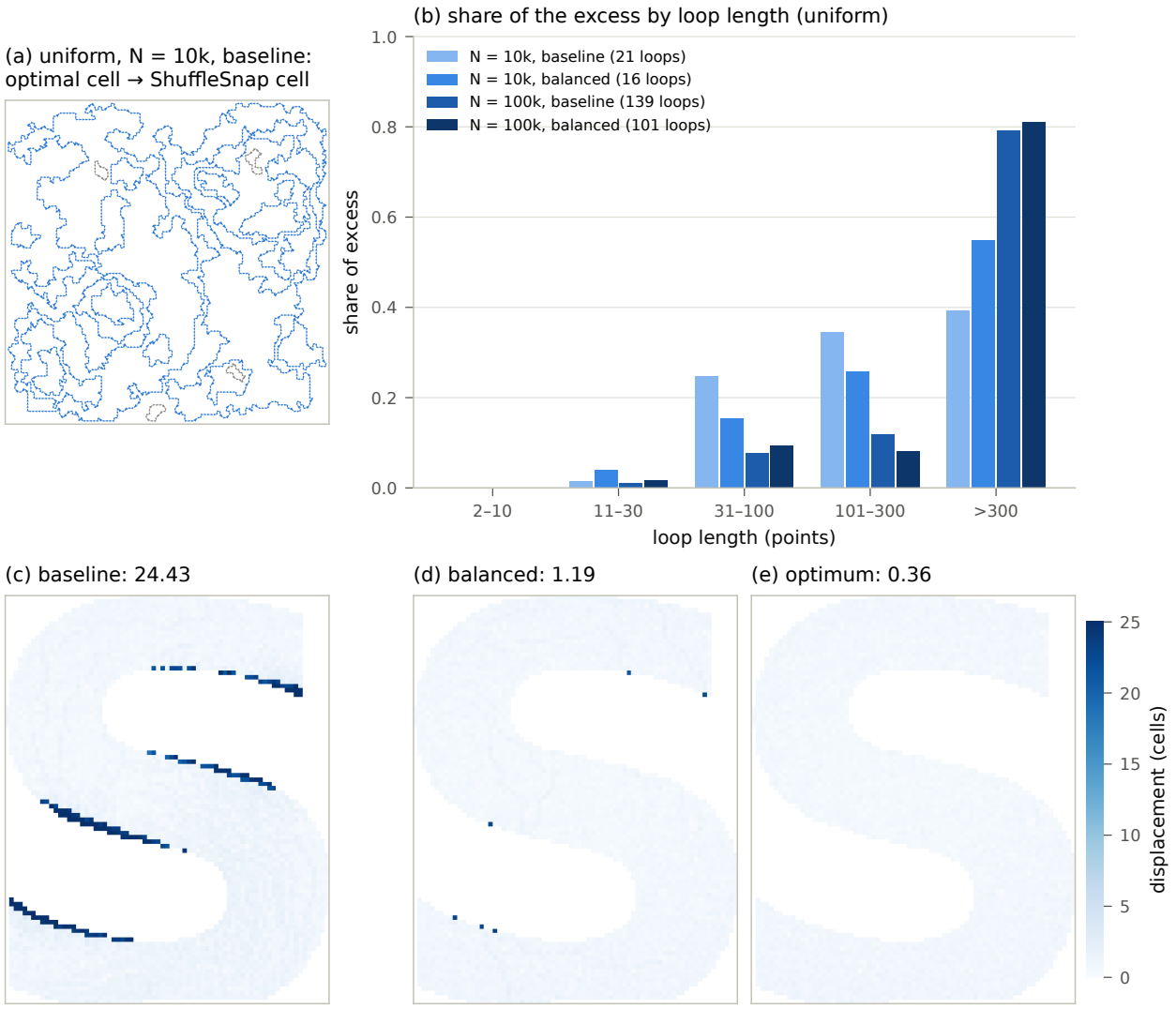


Figure 8: Where ShuffleSnap loses to the optimum (development instances, shown for mechanism rather than as claims). (a) 10,000 uniform points, baseline. Each arrow goes from a point's optimal cell to its ShuffleSnap cell. The two layouts differ by closed loops of one-cell moves; loops longer than 30 points are blue. (b) Share of the excess carried by loops of each length: at 10^5 points, about 80% of it sits in loops of more than 300 points. (c–e) A jittered lattice (optimum ≈ identity) on the S-shaped mask. The baseline routes a stripe of points across the concave bays (displacement ≈25 cells); the balanced preset leaves a few stray points; the optimum moves every point less than a cell. Numbers are mean squared displacement.

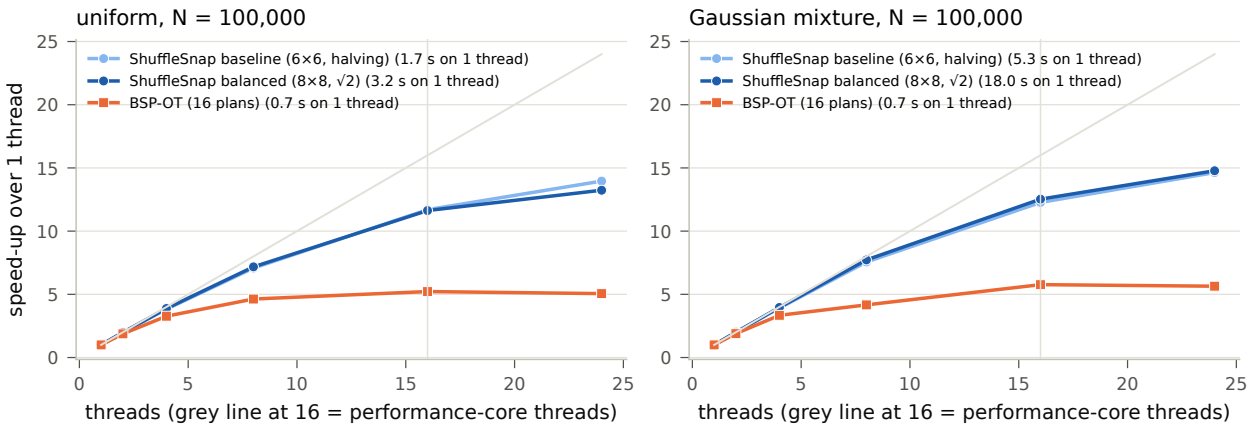


Figure 9: Speed-up over one thread at $N = 100{,}000$ (median of one run per setting, threads not pinned). The machine has 16 performance-core and 8 efficiency-core hardware threads; the light diagonal is linear speed-up. ShuffleSnap's windows are independent within a phase, so it keeps scaling with the efficiency cores. BSP-OT stops improving at 8–16 threads, because its plans run in parallel but the merge that combines them is largely sequential.

$w \in \{6, 8, 12\}$, a ring larger than the window was never touched (0 points moved), so the result has cost L against an optimum of 0. A ring of side at most $w/2 + 1$ always lies inside one of the four shifted tilings and was repaired. A 6×6 ring survived 6×6 windows when it straddled the borders of all four tilings. Relative excess over the optimum is therefore unbounded for this method, and the package's test suite includes this configuration as a documented limitation.

On real inputs, larger windows remove the shorter loops, and the $\sqrt{2}$ schedule creates fewer loops in the first place, but the long loops remain. Their number and total length grow with N, which is why the baseline's relative excess on uniform points grows from about 1% at 10^3 points to 2.5% at 10^5 and 5.3% at 10^6 (Section 7.4).

Masks make the same mechanism visible at a larger scale (Figure 8c–e). At a coarse stride, a window's cells can lie on both sides of a concave bay, so mass gets routed across the bay. By the time the stride is 1, fixing the routing would require moving a whole arm's worth of points along the arm, which no window can do. With a random start, the baseline left a stripe of points displaced by about 25 cells. The finer $\sqrt{2}$ schedule revisits intermediate scales and mostly avoids the error; the held-out numbers are in Section 7.6.

7.9 Parallel scaling

Figure 9 shows parallel scaling. ShuffleSnap doubles its speed with each doubling of threads up to 8 (7.7× at 8 threads). It reaches 12.5× at 16 and 14.8× at 24, with the same assignment at every thread count. BSP-OT reaches about 5–6× at 16 threads and gains nothing from the efficiency cores. We did not profile where ShuffleSnap loses efficiency beyond 8 threads. The efficiency cores are slower than the performance cores, and each phase ends with a synchronization point.

8 Discussion and limitations

What the hypothesis survived. Coarse-to-fine strided windows get very close to the optimum for the smooth, clustered layouts that motivate the problem. On the two real 70,000-point UMAP embeddings, the default preset is within 0.002% of the certified optimum and visually indistinguish-

able from it. The method needs no dense matrix, runs in 0.2–1.4 s with all 24 threads at 10^5 points (balanced preset), gives the same result for any number of threads, and never increases the cost.

What it did not survive. ShuffleSnap is a local search, and its failures follow from that. (i) On near-uniform data, its relative excess grows with N (for the baseline, from about 1% at 10^3 points to 2.5% at 10^5 and 5.3% at 10^6). The excess consists of a few long closed loops of one-cell shifts that no window can see whole. (ii) Masks with concave bays and grids with surplus cells need global rerouting of mass, which the baseline gets wrong. With uniform points on the S-shaped mask it was 18% above optimal, worse than BSP-OT. The $\sqrt{2}$ schedule removes most of this (1.3% for balanced, 0.13% for quality), but not all of it. (iii) A good start does not help, and structured starts hurt. (iv) On a jittered lattice, where the optimum is almost the identity, every heuristic we tested stays far from optimal in relative terms.

The exact solver changes the question. We built the certified solver as an evaluation tool. It turned out to be practical. Starting from ShuffleSnap's answer, it proved optimality at 10^5 points in 7 s to 23 min on one core (median about 2 min), and at 10^6 uniform points in 12 min. When a certified optimum is worth the wait, it is the better choice at this scale, and ShuffleSnap's role is then that of a fast, good-enough answer, a warm start, and a method for sizes and time budgets where exact solving is out of reach. We report this even though it narrows the case for the method we set out to test.

Displacement is not neighbourhood. On the real embeddings, BSP-OT comes within 2–9% of the optimal displacement, yet its layouts are locally scrambled (Figure 5, row 2), and it halves both neighbourhood measures. RasterFairy keeps local order inside its blocks and breaks it along block seams. Even the optimum tears neighbourhoods along cluster boundaries (Figure 5, row 4). If neighbourhood preservation is the goal, it should be optimized or at least measured directly, e.g. with LAS/FLAS and DPQ [2]. A lower displacement does not guarantee it.

Which method to use. In our measurements:

- If a proven optimum is worth minutes of computation, use the certified solver started from ShuffleSnap. We tested it up to 10^6 uniform and 2×10^5 clustered points.
- For interactive use, the balanced (or fast) preset gives near-optimal layouts of 10^5 points in about a second on a multicore machine.
- Under very tight budgets (below about 1 s on one core at 10^5 points), BSP-OT gives the best result available.
- When the position of empty cells does not matter, an exact-fit mask is both simpler and handled far better than surplus cells, by every method.

Limitations of this study. Everything ran on one workstation, CPU only. BSP-OT has a CUDA implementation that we did not test. The objective is squared Euclidean displacement in 2D. The window solver works for any cost, but we did not evaluate other costs or 3D grids. The certified solver depends on OR-Tools and on integer scaling. Its certificate accounts for the scaling, but its running time depends on the candidate set and the warm start. Real data consist of two UMAP embeddings with one parameter setting. The bbox normalization is part of the problem definition; other normalizations change the optimum. Timings exclude warm-up (imports, thread

start-up) and follow one pinning policy, and unrelated background services were running (load about 1). We compared against the representative settings of each comparator, but not every setting. In particular, RasterFairy's swap optimizer was excluded: it needs dense $N \times N$ memory, and its PyPI release does not run on Python 3.

9 Conclusion

Treated as a hypothesis, the Shellsort idea largely held up. Strided exact windows, refined from coarse to fine, bring clustered layouts, including two real UMAP embeddings of 70,000 points, to within hundredths of a percent of a proven optimum in about a second, with little memory and deterministic parallelism. The idea also has sharp limits. Its remaining error forms long loops that no window can see, which makes uniform data, lattices, concave masks and surplus cells hard, with an unbounded worst case. A finer $\sqrt{2}$ stride schedule with larger windows is a better default than the original halving schedule. The most consequential finding may be the exact solver built to judge the heuristic: seeded with ShuffleSnap's answer, proven optima are within reach at 10^5–10^6 points. For point-to-grid layout, exactness is therefore a matter of minutes, not a matter of impossibility.

References

[1] Franz Aurenhammer, Friedrich Hoffmann, and Boris Aronov. Minkowski-type theorems and least-squares clustering. *Algorithmica*, 20(1):61–76, 1998. doi: 10.1007/PL00009187. URL `https://doi.org/10.1007/PL00009187`.

[2] Kai Uwe Barthel, Nico Hezel, Klaus Jung, and Konstantin Schall. Improved evaluation and generation of grid layouts using distance preservation quality and linear assignment sorting. *Computer Graphics Forum*, 42(1):261–276, 2023. doi: 10.1111/cgf.14718. URL `https://doi.org/10.1111/cgf.14718`.

[3] Dimitri P. Bertsekas. A distributed algorithm for the assignment problem. Working paper, Laboratory for Information and Decision Systems, Massachusetts Institute of Technology, Cambridge, MA, March 1979. URL `https://web.mit.edu/dimitrib/www/Bertsekas_Auction_Distributed_1979.pdf`. Original auction algorithm. Retyped copy hosted on the author's MIT page.

[4] Dimitri P. Bertsekas. The auction algorithm: A distributed relaxation method for the assignment problem. *Annals of Operations Research*, 14(1):105–123, 1988. doi: 10.1007/BF02186476. URL `https://doi.org/10.1007/BF02186476`.

[5] Nicolas Bonneel, Julien Rabin, Gabriel Peyré, and Hanspeter Pfister. Sliced and Radon Wasserstein barycenters of measures. *Journal of Mathematical Imaging and Vision*, 51(1):22–45, 2015. doi: 10.1007/s10851-014-0506-3. URL `https://doi.org/10.1007/s10851-014-0506-3`.

[6] Yann Brenier. Polar factorization and monotone rearrangement of vector-valued functions. *Communications on Pure and Applied Mathematics*, 44(4):375–417, 1991. doi: 10.1002/cpa.3160440402. URL `https://doi.org/10.1002/cpa.3160440402`.

[7] Rainer Burkard, Mauro Dell'Amico, and Silvano Martello. *Assignment Problems*. Society for Industrial and Applied Mathematics, 2009. ISBN 978-0-89871-663-4. doi: 10.1137/1.9780898717754. URL `https://doi.org/10.1137/1.9780898717754`. First edition. A revised reprint appeared in 2012 (see burkard2012assignment).

[8] David F. Crouse. On implementing 2D rectangular assignment algorithms. *IEEE Transactions on Aerospace and Electronic Systems*, 52(4):1679–1696, 2016. doi: 10.1109/TAES.2016.140952. URL https://doi.org/10.1109/TAES.2016.140952.

[9] Marco Cuturi. Sinkhorn distances: Lightspeed computation of optimal transport. In C. J. Burges, L. Bottou, M. Welling, Z. Ghahramani, and K. Weinberger, editors, *Advances in Neural Information Processing Systems*, volume 26. Curran Associates, Inc., 2013. URL https://proceedings.neurips.cc/paper/2013/hash/af21d0c97db2e27e13572cbf59eb343d-Abstract.html.

[10] Rémi Flamary, Cédric Vincent-Cuaz, Nicolas Courty, Alexandre Gramfort, Oleksii Kachaiev, Huy Quang Tran, Laurène David, Clément Bonet, Nathan Cassereau, Théo Gnassounou, Eloi Tanguy, Julie Delon, Antoine Collas, Sonia Mazelet, Laetitia Chapel, Tanguy Kerdoncuff, Xizheng Yu, Matthew Feickert, Paul Krzakala, Tianlin Liu, Eduardo Fernandes Montesuma, Nathan Neike, Baptiste Genest, David Coeurjolly, Thibaut Germain, Sienna O'Shea, Marco Corneli, and Ferdinand Genans. POT Python optimal transport. https://github.com/PythonOT/POT, 2026. URL https://github.com/PythonOT/POT. Version 0.9.7. Accessed 2026-09-23.

[11] Ohad Fried, Stephen DiVerdi, Maciej Halber, Elena Sizikova, and Adam Finkelstein. IsoMatch: Creating informative grid layouts. *Computer Graphics Forum*, 34(2):155–166, 2015. doi: 10.1111/cgf.12549. URL https://doi.org/10.1111/cgf.12549. Proc. Eurographics 2015.

[12] Baptiste Genest, Nicolas Bonneel, Vincent Nivoliers, and David Coeurjolly. BSP-OT: Sparse transport plans between discrete measures in loglinear time. *ACM Transactions on Graphics*, 44(6):1–15, December 2025. doi: 10.1145/3763281. URL https://doi.org/10.1145/3763281. Proc. SIGGRAPH Asia 2025.

[13] Baptiste Genest, Nicolas Bonneel, Vincent Nivoliers, and David Coeurjolly. BSP-OT: Official source code. https://github.com/baptiste-genest/BSP-OT, 2025. URL https://github.com/baptiste-genest/BSP-OT. C++ (MIT license). A CUDA version is at https://github.com/baptiste-genest/BSP-OT-CUDA. Accessed 2026-09-23.

[14] Samuel Gerber and Mauro Maggioni. Multiscale strategies for computing optimal transport. *Journal of Machine Learning Research*, 18(72):1–32, 2017. URL https://jmlr.org/papers/v18/16-108.html.

[15] Andrew V. Goldberg and Robert Kennedy. An efficient cost scaling algorithm for the assignment problem. *Mathematical Programming*, 71(2):153–177, 1995. doi: 10.1007/BF01585996. URL https://doi.org/10.1007/BF01585996.

[16] Google OR-Tools. ortools/graph/min_cost_flow.h: Source file (OR-Tools v9.15 C++ reference). https://or-tools.github.io/docs/cpp/min__cost__flow_8h_source.html. URL https://or-tools.github.io/docs/cpp/min__cost__flow_8h_source.html. States: "An implementation of a cost-scaling push-relabel algorithm for the min-cost flow problem." The companion linear_assignment.h states a cost-scaling push-relabel algorithm for the assignment problem "from the paper of Goldberg and Kennedy (1995)". Accessed 2026-09-23.

[17] Gladys M. Hilasaca, Wilson E. Marcílio-Jr, Danilo M. Eler, Rafael M. Martins, and Fernando V. Paulovich. A grid-based method for removing overlaps of dimensionality reduction scatterplot layouts. *IEEE Transactions on Visualization and Computer Graphics*, 30(8):5733–5749, 2024. doi: 10.1109/TVCG.2023.3309941. URL https://doi.org/10.1109/TVCG.2023.3309941.

[18] R. Jonker and A. Volgenant. A shortest augmenting path algorithm for dense and sparse linear assignment problems. *Computing*, 38(4):325–340, 1987. doi: 10.1007/BF02278710. URL https://doi.org/10.1007/BF02278710.

[19] Tomas Kazmar et al. lap: Linear assignment problem solver (LAPJV/LAPMOD). https://github.com/gatagat/lap. URL https://github.com/gatagat/lap. Python package (BSD-2-Clause). Latest PyPI version 0.5.13 (2026-02-23). Accessed 2026-09-23.

[20] Daniel A. Keim and Annemarie Herrmann. The Gridfit algorithm: An efficient and effective approach to visualizing large amounts of spatial data. In *Proceedings Visualization '98*, pages 181–188. IEEE, 1998. doi: 10.1109/VISUAL.1998.745301. URL https://doi.org/10.1109/VISUAL.1998.745301.

[21] Mario Klingemann. RasterFairy. https://github.com/Quasimondo/RasterFairy, 2015. URL https://github.com/Quasimondo/RasterFairy. GitHub repository created 2016-01-22; latest PyPI release of `rasterfairy` is 1.0.6 (2019-04-29). Accessed 2026-09-23.

[22] Teuvo Kohonen. Self-organized formation of topologically correct feature maps. *Biological Cybernetics*, 43(1):59–69, 1982. doi: 10.1007/BF00337288. URL https://doi.org/10.1007/BF00337288.

[23] Harold W. Kuhn. The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1–2):83–97, 1955. doi: 10.1002/nav.3800020109. URL https://doi.org/10.1002/nav.3800020109.

[24] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. doi: 10.1109/5.726791. URL https://doi.org/10.1109/5.726791.

[25] Bruno Lévy. A numerical algorithm for L_2 semi-discrete optimal transport in 3D. *ESAIM: Mathematical Modelling and Numerical Analysis*, 49(6):1693–1715, 2015. doi: 10.1051/m2an/2015055. URL https://doi.org/10.1051/m2an/2015055.

[26] Vadim Markovtsev. Jonker-Volgenant algorithm + t-SNE = super powers. source{d} blog. Archived at http://web.archive.org/web/20180611012448/https://blog.sourced.tech/post/lapjv/, March 2017. URL http://web.archive.org/web/20180611012448/https://blog.sourced.tech/post/lapjv/. Posted 14 March 2017. Accessed 2026-09-23 via the Internet Archive.

[27] Vadim Markovtsev and source{d}. lapjv: Linear assignment problem solver using Jonker-Volgenant algorithm. https://github.com/src-d/lapjv, 2017. URL https://github.com/src-d/lapjv. Python 3 module (MIT license). First PyPI release 1.0.0 (2017-01-09); latest PyPI version 1.3.29 (2026-09-22). Accessed 2026-09-23.

[28] Kyle McDonald. CloudToGrid. https://github.com/kylemcdonald/CloudToGrid, 2016. URL https://github.com/kylemcdonald/CloudToGrid. Jupyter notebook mapping a t-SNE point cloud to a 2D grid via the assignment problem (lapjv and hungarian packages). Accessed 2026-09-23.

[29] Leland McInnes, John Healy, and James Melville. UMAP: Uniform manifold approximation and projection for dimension reduction, 2018. URL https://arxiv.org/abs/1802.03426.

[30] Leland McInnes, John Healy, Nathaniel Saul, and Lukas Großberger. UMAP: Uniform manifold approximation and projection. *Journal of Open Source Software*, 3(29):861, 2018. doi: 10.21105/joss.00861. URL https://doi.org/10.21105/joss.00861.

[31] Quentin Mérigot. A multiscale approach to optimal transport. *Computer Graphics Forum*, 30(5):1583–1592, 2011. doi: 10.1111/j.1467-8659.2011.02032.x. URL https://doi.org/10.1111/j.1467-8659.2011.02032.x.

[32] James Munkres. Algorithms for the assignment and transportation problems. *Journal of the Society for Industrial and Applied Mathematics*, 5(1):32–38, 1957. doi: 10.1137/0105003. URL https://doi.org/10.1137/0105003.

[33] Laurent Perron and Vincent Furnon. OR-Tools. https://developers.google.com/optimization/, 2026. URL https://developers.google.com/optimization/. Version v9.15, released 2026-01-12. Accessed 2026-09-23.

[34] Gabriel Peyré and Marco Cuturi. Computational optimal transport with applications to data sciences. *Foundations and Trends in Machine Learning*, 11(5–6):355–607, 2019. doi: 10.1561/2200000073. URL https://doi.org/10.1561/2200000073.

[35] Novi Quadrianto, Le Song, and Alex J. Smola. Kernelized sorting. In D. Koller, D. Schuurmans, Y. Bengio, and L. Bottou, editors, *Advances in Neural Information Processing Systems*, volume 21. Curran Associates, Inc., 2008. URL https://proceedings.neurips.cc/paper_files/paper/2008/hash/94c7bb58efc3b337800875b5d382a072-Abstract.html.

[36] Novi Quadrianto, Alex J. Smola, Le Song, and Tinne Tuytelaars. Kernelized sorting. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 32(10):1809–1821, 2010. doi: 10.1109/TPAMI.2009.184. URL https://doi.org/10.1109/TPAMI.2009.184.

[37] Julien Rabin, Gabriel Peyré, Julie Delon, and Marc Bernot. Wasserstein barycenter and its application to texture mixing. In Alfred M. Bruckstein, Bart M. ter Haar Romeny, Alexander M. Bronstein, and Michael M. Bronstein, editors, *Scale Space and Variational Methods in Computer Vision (SSVM 2011)*, volume 6667 of *Lecture Notes in Computer Science*, pages 435–446, Berlin, Heidelberg, 2012. Springer. doi: 10.1007/978-3-642-24785-9_37. URL https://doi.org/10.1007/978-3-642-24785-9_37.

[38] Sharath Raghvendra and Pankaj K. Agarwal. A near-linear time ε-approximation algorithm for geometric bipartite matching. *Journal of the ACM*, 67(3):1–19, 2020. doi: 10.1145/3393694. URL https://doi.org/10.1145/3393694.

[39] Bernhard Schmitzer. A sparse multiscale algorithm for dense optimal transport. *Journal of Mathematical Imaging and Vision*, 56(2):238–259, 2016. doi: 10.1007/s10851-016-0653-9. URL https://doi.org/10.1007/s10851-016-0653-9.

[40] R. Sharathkumar and Pankaj K. Agarwal. Algorithms for the transportation problem in geometric settings. In *Proceedings of the Twenty-Third Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 306–317. Society for Industrial and Applied Mathematics, 2012. doi: 10.1137/1.9781611973099.29. URL https://doi.org/10.1137/1.9781611973099.29.

[41] Donald L. Shell. A high-speed sorting procedure. *Communications of the ACM*, 2(7):30–32, 1959. doi: 10.1145/368370.368387. URL https://doi.org/10.1145/368370.368387.

[42] Grant Strong and Minglun Gong. Self-sorting map: An efficient algorithm for presenting multimedia data in structured layouts. *IEEE Transactions on Multimedia*, 16(4):1045–1058, 2014. doi: 10.1109/TMM.2014.2306183. URL https://doi.org/10.1109/TMM.2014.2306183.

[43] Kenshi Takayama. bspot-python: Python binding to BSP-OT. https://github.com/kenshi84/bspot-python, 2026. URL https://github.com/kenshi84/bspot-python. PyPI package `bspot`, latest version 1.2.0 (2026-01-28), MIT license. Accessed 2026-09-23.

[44] Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9(86):2579–2605, 2008. URL https://jmlr.org/papers/v9/vandermaaten08a.html.

[45] Jarkko Venna and Samuel Kaski. Neighborhood preservation in nonlinear projection methods: An experimental study. In Georg Dorffner, Horst Bischof, and Kurt Hornik, editors, *Artificial Neural Networks — ICANN 2001*, volume 2130 of *Lecture Notes in Computer Science*, pages 485–491, Berlin, Heidelberg, 2001. Springer. doi: 10.1007/3-540-44668-0_68. URL https://doi.org/10.1007/3-540-44668-0_68.

[46] Jarkko Venna and Samuel Kaski. Local multidimensional scaling. *Neural Networks*, 19(6–7):889–899, 2006. doi: 10.1016/j.neunet.2006.05.014. URL https://doi.org/10.1016/j.neunet.2006.05.014.

[47] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, C J Carey, İlhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M. Archibald, Antônio H. Ribeiro, Fabian Pedregosa, Paul van Mulbregt, and SciPy 1.0 Contributors. SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3):261–272, 2020. doi: 10.1038/s41592-019-0686-2. URL https://doi.org/10.1038/s41592-019-0686-2.

[48] Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms, 2017. URL https://arxiv.org/abs/1708.07747.

A Reproducing the results

The paper repository contains every script, raw result, and figure source. The implementation is a separate repository (`shufflesnap`). Starting from a checkout of both side by side:

```
pip install ./shufflesnap[exact]   # C++ build, ~2 s
pip install -r shufflesnap-paper/requirements.txt
cd shufflesnap-paper
python experiments/make_embeddings.py mnist fashion   # UMAP, fixed seed
python experiments/references_all.py 6                # certified optima (parallel)
bash experiments/run_all.sh                           # all benchmarks, sequential
python experiments/small_exactness.py                 # small-N exactness table
python experiments/dev_ablation.py                    # development-set search
python experiments/make_figures.py all && python experiments/make_tables.py
```

```
python experiments/fig_method.py; python experiments/fig_failures.py
python experiments/fig_visual.py mnist; python experiments/fig_visual.py fashion
python experiments/fig_masks.py
cd paper && tectonic main.tex
```

Raw results are JSON lines in `results/raw/`: one record per run, holding the full run specification, timings, memory, validity check, cost, displacement statistics, and the software environment (library versions, commit hashes of the comparator repositories). Certified optima are cached in `results/reference/` with their lower bound and gap. The lab notebook `notebook/LOG.md` records decisions and negative results in the order they happened.

Timing boundary (summary). A timed run is one call of a method adapter in a fresh process, after a warm-up call on a 256-point instance. It includes method-specific preprocessing and excludes instance generation, the shared normalization, and validation. Peak memory is the rise of peak RSS during the call.

B Additional tables

Table 7: Absolute excess over the certified optimum, in cells^2 per point, at $N = 100{,}000$ (same runs as Table 2). "Optimum" is the certified optimal mean squared displacement. Relative and absolute excess tell different stories on the heavy-tailed input, where outliers dominate the optimum, and on the jittered lattice, where the optimum is tiny.

distribution	optimum	SS base	SS fast	SS bal.	SS qual.	BSP 16	BSP 64	RF master	RF PyPI	Bisect.
uniform	1.80	0.046	0.027	0.015	0.008	0.80	0.58	0.80	1.10	1.81
Gaussian	6502	0.031	0.021	0.011	0.005	3.61	2.60	0.38	0.66	1.17
Gaussian mixture	4441	0.53	0.39	0.11	0.010	252	167	751	752	554
90/10 clusters	8743	2.69	2.99	1.01	0.079	2.77	1.66	102	101	49.9
ring	1731	0.30	0.047	0.022	0.008	20.1	16.5	710	710	697
spiral	1338	0.31	0.17	0.055	0.015	133	108	366	367	460
checkerboard	32.1	0.055	0.029	0.018	0.006	16.0	11.6	137	137	145
heavy tails	32645	0.009	0.009	0.007	0.005	0.019	0.012	0.018	0.070	0.059
duplicates	4140	3.92	3.31	1.45	0.47	491	228	2034	2035	1645
near-line	9876	0.96	0.75	0.21	0.033	0.81	0.20	1912	1912	1884
jittered lattice	0.43	0.14	0.097	0.054	0.031	0.27	0.19	0.056	0.12	1.20

Table 8: As Table 2, at $N = 10{,}000$. Here the dense SciPy solver is feasible; the last column gives its time on instance seed 1 (it returns the same optimum as the certified solver).

distribution	SS base	SS fast	SS bal.	SS qual.	BSP 16	BSP 64	RF master	RF PyPI	Bisect.	exact (s)	dense (s)
uniform	1.72	1.09	0.69	0.28	27.6	17.9	31.8	82.4	86.2	0.71	10.8
Gaussian	0.003	0.002	0.001	<0.001	0.13	0.093	0.040	0.14	0.15	2.43	205
Gaussian mixture	0.016	0.011	0.004	<0.001	3.78	2.54	38.7	39.7	13.1	2.98	176
90/10 clusters	0.036	0.025	0.011	0.001	0.13	0.049	1.32	1.47	0.66	4.08	227
ring	0.030	0.012	0.007	0.005	3.78	1.56	40.5	41.6	39.4	2.64	104
spiral	0.043	0.026	0.013	0.005	10.3	7.30	24.6	29.9	31.9	2.97	84.9
checkerboard	0.79	0.50	0.28	0.090	55.4	44.8	271.4	260.7	265.2	0.73	15.9
heavy tails	<0.001	<0.001	<0.001	<0.001	0.005	0.003	<0.001	0.002	0.003	4.56	352
duplicates	0.12	0.084	0.045	0.014	12.0	4.97	57.6	58.0	36.6	1.55	126
near-line	0.016	0.005	0.003	<0.001	0.027	0.007	19.4	19.8	19.5	7.74	246
jittered lattice	26.1	17.8	11.4	6.19	65.2	49.4	12.0	33.6	245.5	0.34	2.88
median time (s)	0.35	0.44	0.96	2.96	0.063	0.25	0.22	0.029	0.003	2.64	126

Table 9: As Table 2, at $N = 50{,}000$.

distribution	SS base	SS fast	SS bal.	SS qual.	BSP 16	BSP 64	RF master	RF PyPI	Bisect.	exact (s)
uniform	3.61	2.24	1.20	0.48	45.5	36.8	40.4	65.4	114.2	6.82
Gaussian	<0.001	<0.001	<0.001	<0.001	0.056	0.039	0.009	0.018	0.031	38.4
Gaussian mixture	0.012	0.008	0.001	<0.001	5.47	3.83	38.2	38.2	14.6	47.6
90/10 clusters	0.031	0.026	0.005	<0.001	0.049	0.025	1.21	1.22	0.86	65.1
ring	0.017	0.004	0.002	<0.001	2.15	0.59	41.1	41.1	42.2	37.6
spiral	0.025	0.014	0.006	0.002	11.4	7.74	27.3	27.4	25.0	43.1
checkerboard	0.29	0.16	0.085	0.035	81.6	69.4	398.0	400.1	423.4	14.4
heavy tails	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	92.1
duplicates	0.11	0.083	0.038	0.013	13.8	4.91	58.6	58.6	39.7	151
near-line	0.007	0.002	<0.001	<0.001	0.028	0.003	19.7	19.7	19.5	77.9
jittered lattice	24.5	17.3	13.1	6.91	59.4	41.2	12.1	27.7	205.6	2.87
median time (s)	2.26	3.04	6.95	22.6	0.34	1.36	1.45	12.4	0.016	43.1

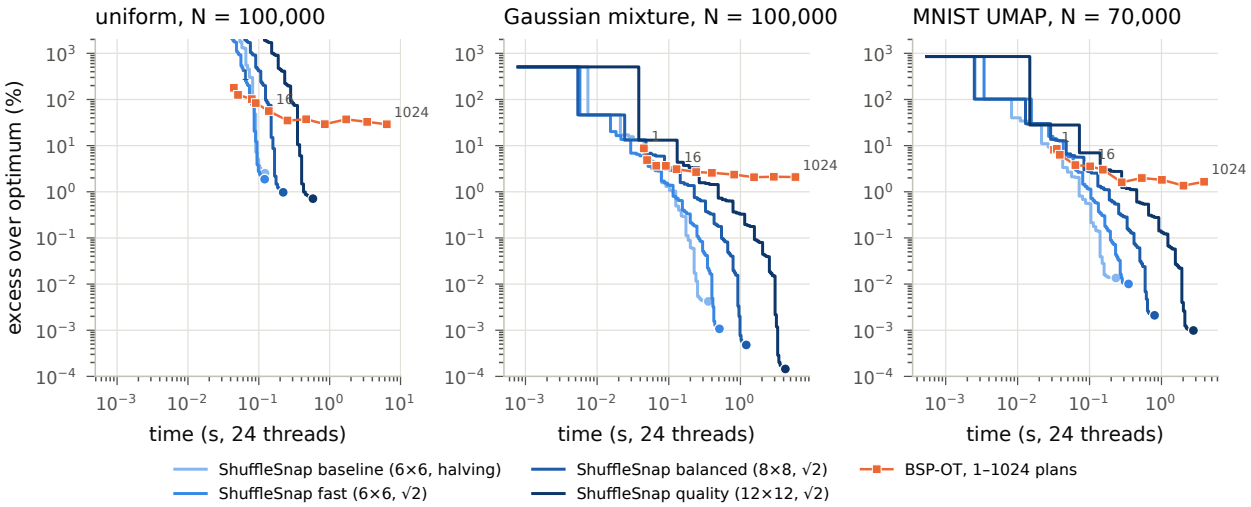


Figure 10: As Figure 3, with 24 threads for ShuffleSnap and BSP-OT (RasterFairy is single-threaded).

Table 10: All held-out ablations (24 threads): mean excess over the certified optimum (%) and median time (s) over instance seeds 1–3. Each row changes one setting of the baseline (6×6 windows, stride ratio 2, four tilings, random start, one round per level, greedy-start Hungarian solver, unchanged windows skipped, automatic start stride).

change	setting	uniform 100k		mixture 100k		MNIST 70k	
		excess %	time (s)	excess %	time (s)	excess %	time (s)
window side (ratio 2)	3×3	15.7	0.054	0.16	0.074	0.21	0.051
	4×4	6.36	0.072	0.035	0.12	0.079	0.084
	5×5	3.71	0.098	0.022	0.22	0.031	0.14
	6×6	2.51	0.13	0.011	0.37	0.015	0.24
	7×7	1.94	0.19	0.006	0.58	0.013	0.39
	8×8	1.52	0.25	0.004	0.87	0.004	0.59
	10×10	0.93	0.44	0.001	1.80	0.002	1.19
	12×12	0.54	0.68	<0.001	3.16	0.001	2.12
	16×16	0.29	1.47	<0.001	8.43	<0.001	5.72
window side (ratio $\sqrt{2}$)	4×4	3.76	0.072	0.027	0.17	0.068	0.11
	6×6	1.50	0.14	0.008	0.53	0.010	0.35
	8×8	0.84	0.25	0.002	1.25	0.002	0.81
	10×10	0.54	0.42	<0.001	2.49	0.002	1.55
	12×12	0.44	0.62	<0.001	4.33	<0.001	2.88
	16×16	0.19	1.32	<0.001	11.1	<0.001	7.78
stride ratio	3	4.14	0.11	0.014	0.32	0.041	0.23
	2 (baseline)	2.51	0.13	0.011	0.37	0.015	0.25
	1.68	1.99	0.13	0.007	0.43	0.011	0.28
	1.41	1.50	0.14	0.008	0.53	0.010	0.35
	1.19	1.58	0.17	0.009	0.80	0.013	0.50
	1.1	1.93	0.20	0.009	1.04	0.018	0.64
start	random (baseline)	2.51	0.14	0.011	0.37	0.015	0.24
	bisection	29.9	0.16	0.21	0.73	0.14	0.33
	row sort	22.9	0.13	0.042	1.02	0.12	0.41
rounds per level	1	2.51	0.14	0.011	0.37	0.015	0.24
	2	2.08	0.20	0.009	0.55	0.012	0.36
	3	1.87	0.22	0.009	0.62	0.011	0.41
tilings per round	1	21166	0.046	4.50	0.079	12.3	0.051
	2 (diagonal)	5.49	0.076	0.029	0.20	0.031	0.14
	4 (baseline)	2.51	0.13	0.011	0.36	0.015	0.24
	9	1.86	0.27	0.006	0.76	0.008	0.51
first stride	automatic (baseline)	2.51	0.12	0.011	0.35	0.015	0.24
	capped at 16	11.7	0.57	0.29	27.2	0.44	8.88
	capped at 8	10.9	0.95	0.27	29.1	0.54	11.5
	capped at 4	12.9	1.21	0.28	29.8	0.62	12.0
	capped at 2	15.7	1.30	0.28	29.6	0.58	11.1
	capped at 1	27.4	1.27	0.28	31.2	0.55	11.8
window solver	Hungarian, 6×6	2.51	0.15	0.011	0.36	0.015	0.24
	Hungarian + greedy start, 6×6	2.51	0.14	0.011	0.37	0.015	0.24
	Jonker–Volgenant, 6×6	2.51	0.29	0.011	1.53	0.015	0.88
	Hungarian, 12×12	0.54	0.80	<0.001	3.13	0.001	2.14
	Hungarian + greedy start, 12×12	0.54	0.68	<0.001	3.21	0.001	2.21
	Jonker–Volgenant, 12×12	0.54	5.27	<0.001	32.6	0.001	20.7
skip unchanged windows	yes (baseline)	2.51	0.14	0.011	0.36	0.015	0.24
	no	2.51	0.21	0.011	0.59	0.015	0.41